Lab 1. Ticket enhancer

Saturday walkthrough. Claude Code plugin.

A vague GitHub issue in. A ready contract out.

25 minutes of typing. Fork setup is already done.

Work from `labs/lab1_enhancer`. Not the repo root.

What you will build

A Claude Code plugin that grooms every open draft ticket in **your** fork, one poll at a time.

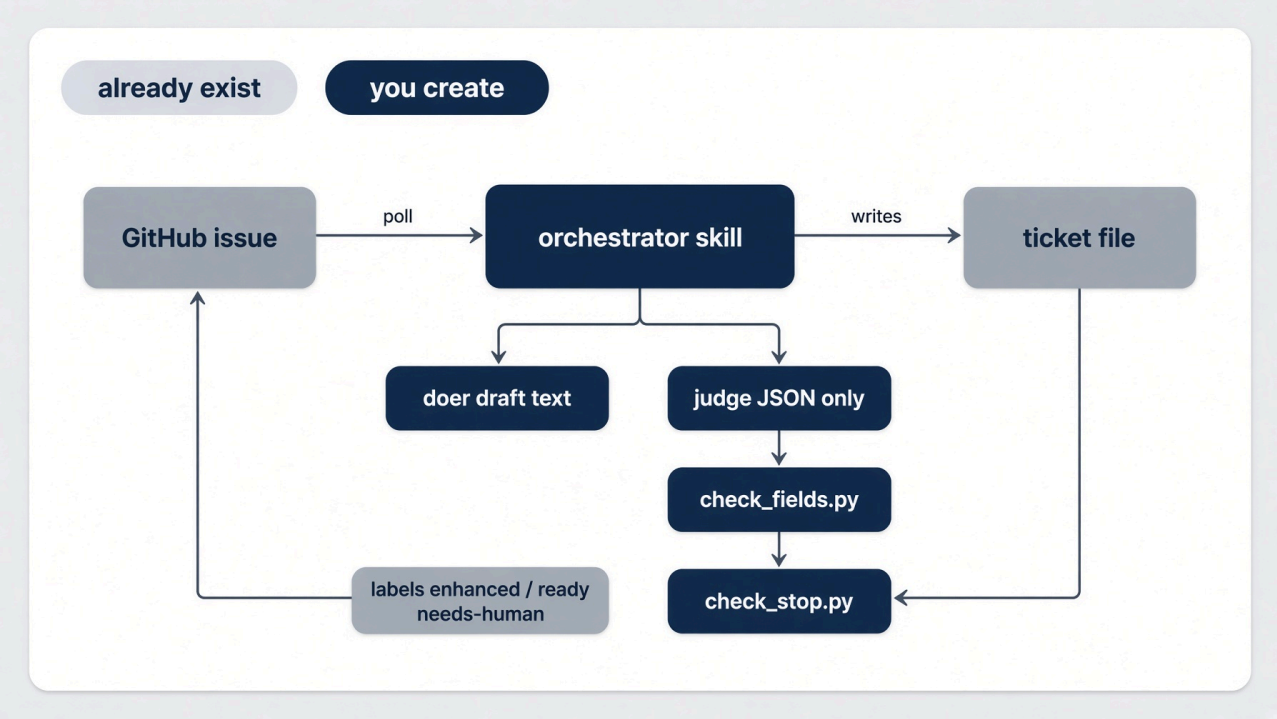
PIECE	PATH YOU CREATE
Judge agent	`.claude/agents/enhancer-judge.md`
Doer agent	`.claude/agents/enhancer-doer.md`
Field check	`.claude/skills/enhancer-loop/scripts/check_fields.py`
Stop check	`.claude/skills/enhancer-loop/scripts/check_stop.py`
Orchestrator skill	`.claude/skills/enhancer-loop/SKILL.md`

Already in the folder: `Taskfile.yml`, `bin/*.sh`, `config.json.example`, `.claude/settings.json`, four prompt files.

The finished answer is `solutions/sol1_enhancer/`. Read `HOW_TO_RUN.md` there after this lab.

Starting architecture

Components that already exist **(gray)**. Components you create **(navy)**.



Trigger lives **outside**: `task run`, `task poll-forever`, `/enhancer-loop`, or Actions. The skill runs **one step and exits**.

Why this lab exists

A prompt that grooms a ticket once is a demo. A loop that polls GitHub, drafts, judges, and stops is a product.

- Nobody sits in chat driving it.
- The judge cannot edit the ticket it grades.
- Ready is arithmetic, not a model claim.
- `task poll-forever` is a seminar lie. Production is an event.

Final outcome. T900, T901, T902 exist as GitHub issues. One `task run` grooms the stubs. A human `LGTM` on a green rubric sets `ready` and `loop: implementer`.

Prerequisites

NEED	CHECK
GitHub account and `gh`	`gh auth status`
Claude Code	`claude --version`
Fork of northwind-field-crm, in **your** account	never `SpillwaveSolutions`
`task`	`task --version`
Working directory	`cd labs/lab1_enhancer`

```
cp config.json.example config.json
# fill fork_owner with your GitHub username
task clone
```

`task clone` writes `../../work/northwind-field-crm`. Paths are built from `TASKFILE_DIR`, so it does not matter where you invoked `task`.

Step 1. Start Claude in this folder

```
cd labs/lab1_enhancer
claude
```

Paste **one prompt at a time** from `prompts/claude-code.md`.

`.claude/settings.json` already denies writes to `loops/`, `scripts/`, and `work/**/tests/**`.

There is no `loops/` package. Duplicate code on purpose.

Step 2. Judge. No write tool

Create `.claude/agents/enhancer-judge.md`.

```
name: enhancer-judge
tools: Read, Grep, Glob
```

Entire final message is one JSON object:

```
{"kind": "bug", "present_fields": ["title", "steps"]}
```

The judge reports `present_fields`. It does **not** compute `missing_fields`. That is Python.

Step 3. Doer. Text only

Create `.claude/agents/enhancer-doer.md`. Same tools. No write.

The orchestrator writes `tickets/<id>.enhancer-candidate.md` from the doer's text. The doer does not write that file.

Investigate `app/` in the target. Use the latest human GitHub comment as the strongest signal.

Step 4. Two Python checks

```
python3 .claude/skills/enhancer-loop/scripts/check_fields.py --demo
python3 .claude/skills/enhancer-loop/scripts/check_stop.py --demo
```

Expected: `all demo assertions passed`.

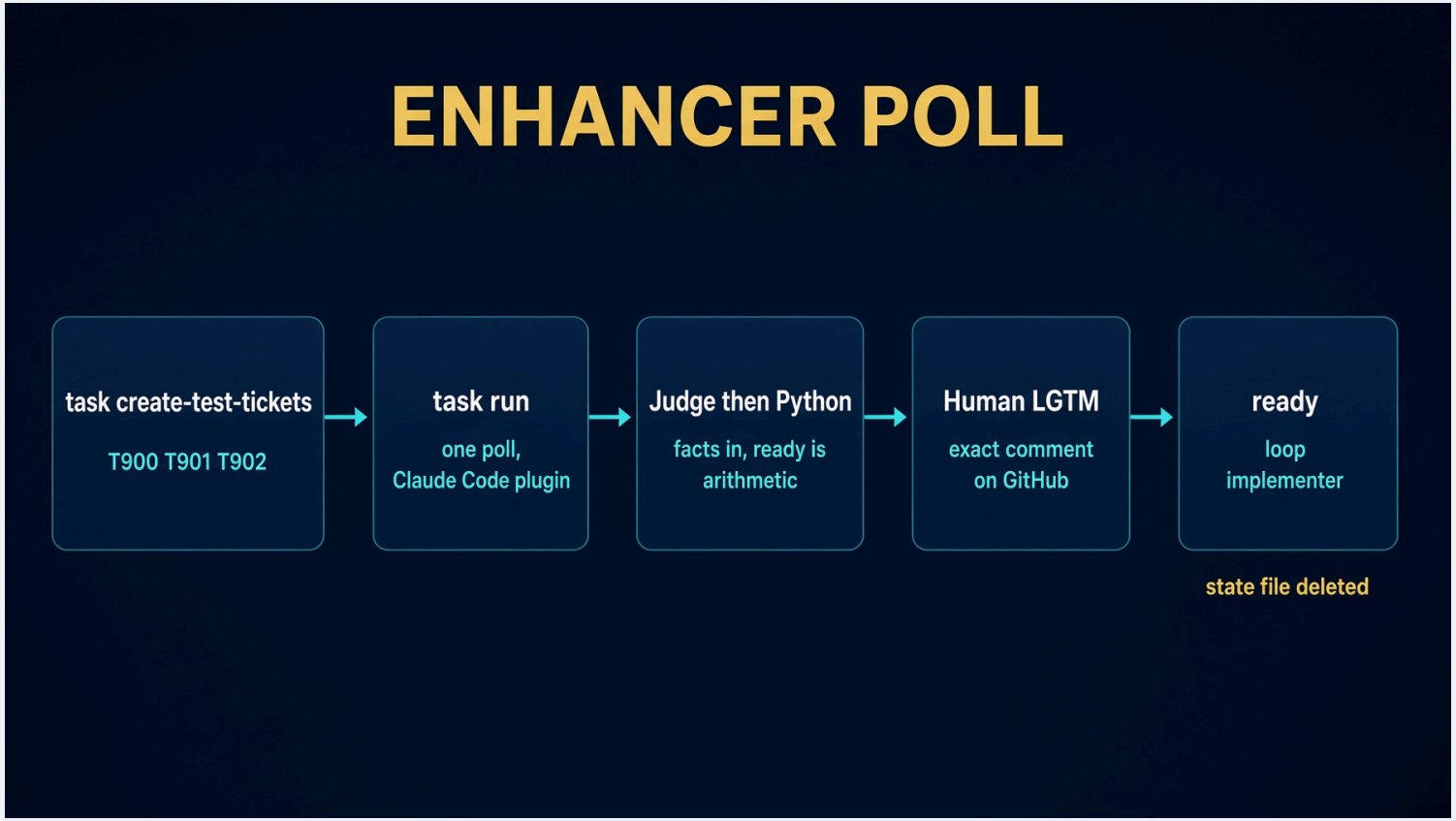
`check_fields.py` computes ready. `check_stop.py` stops on a repeated signature or a spent budget of 3. A model can talk itself past a stop in a prompt. Python will not.

Step 5. Orchestrator skill. Eight rules

`/enhancer-loop --repo <path>`

- A missing comment does not stop you.
- The `enhanced` label is not the work.
- Seed stubs are never ready.
- An issue opened in the GitHub UI is a ticket.
- `ready` comes from `check_fields.py` plus exact `LGTM`.
- `task run` never opens an issue. That is `task create-test-tickets`.
- Comments from the loop carry `<!-- enhancer-loop -->`. Filter the marker, not the author.
- On keep, update the **issue body** to match the ticket, not only the comment.

The poll, in one picture



First poll always runs one round. A ticket with never any comment has nothing for a human to react to yet.

Run one poll

```
task create-test-tickets && task run --
```

ID	KIND	TITLE
T900	bug	Search crashes on an empty query
T901	ui	Add a notes field to the customer page
T902	feature	Export tasks to CSV

`task run` is `claude -p "/enhancer-loop --repo {{.TARGET}}"`.

After poll 1. Then LGTM

Ticket files have real fields. Label `enhanced`. A marked comment. State files under `.harness/`.

Comment `LGTM` on GitHub as yourself, on a ticket `check_fields.py` already calls ready.

Next `task run`:

- label `ready`
- frontmatter `state: ready` and `loop: implementer`
- state file deleted

That is the handoff into Lab 2.

```
task poll-forever --
```

Seminar stand-in. `Ctrl-C` when you are done. Set `poll_interval: "30s"` in `config.json` while testing.

If it looks hung

`claude -p` prints nothing until the run finishes.

```
# config.json: "debug": true
touch debug.log && tail -f debug.log
```

Hooks write one line per tool call. `debug.log` is gitignored. Leave `debug` false for a normal run.

Retest from scratch

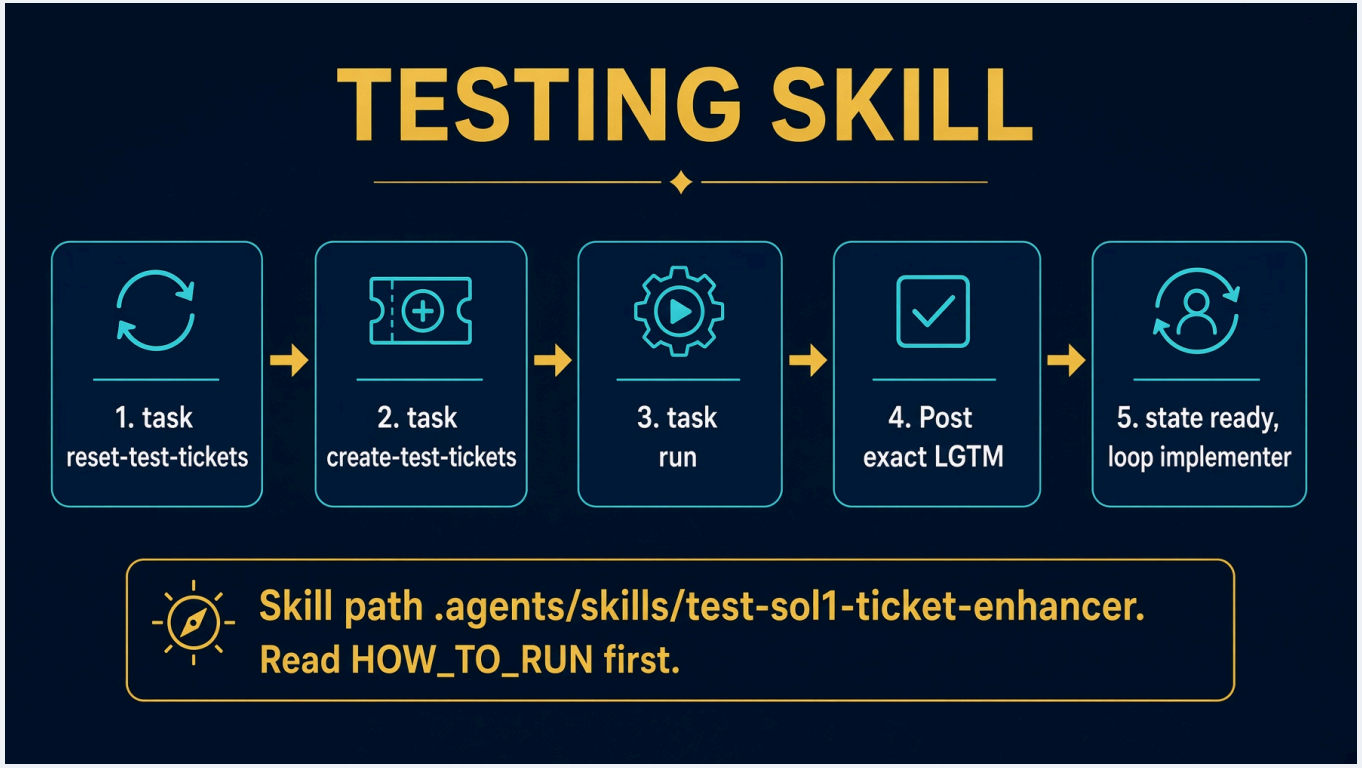
Closing an issue by hand is not a reset. It is the thing that breaks the next poll.

```
task reset-test-tickets
task create-test-tickets
task run --
```

That retires GitHub titles to `[retired-Txxx-...]`, drops `github_issue`, deletes enhancer state, and reseeds T900, T901, T902.

To replay one ticket: keep the issue number, restore draft frontmatter, delete `.harness/last-enhancer-T901.json`, `gh issue reopen`.

Testing skill. After class



`.agents/skills/test-sol1-ticket-enhancer/`

The skill reviews live GitHub issues, posts exact `LGTM`, and checks `state: ready` plus `loop: implementer`. Read that folder's `HOW_TO_RUN.md` first.

Fall behind

Lab 1 is the only lab with a drop-in.

```
cp -R ../../solutions/sol1_enhancer/.claude .claude
```

See `FALL-BEHIND.md`. After class, the finished plugin also has `HOW_TO_RUN.md` and `DESIGN_DOC.md` in `solutions/sol1_enhancer/`.

Recap

Trigger outside. Exits inside. Judge has no write tool. Ready is arithmetic.

`task poll-forever` is a seminar stand-in. Actions is the deploy.

Closing line. The loop is the product. The prompt is not.

Spillwave Solutions | spillwave.com

Lab 2. Ticket implementer and the harness

The centre of the workshop. 55 minutes. Never cut.

A ready ticket in. A green rubric out.

25 minutes of typing. Fill `harness.py`. Nothing else.

Work from `labs/lab2_implementer`.

Why this lab exists

The loop you just built will lie to you. Edit forever. Declare victory on red. Stuff the window.

A true story from this repo: seven tests, green on every run, testing the wrong tree. The conftest put the finished answer on `sys.path` ahead of the work copy.

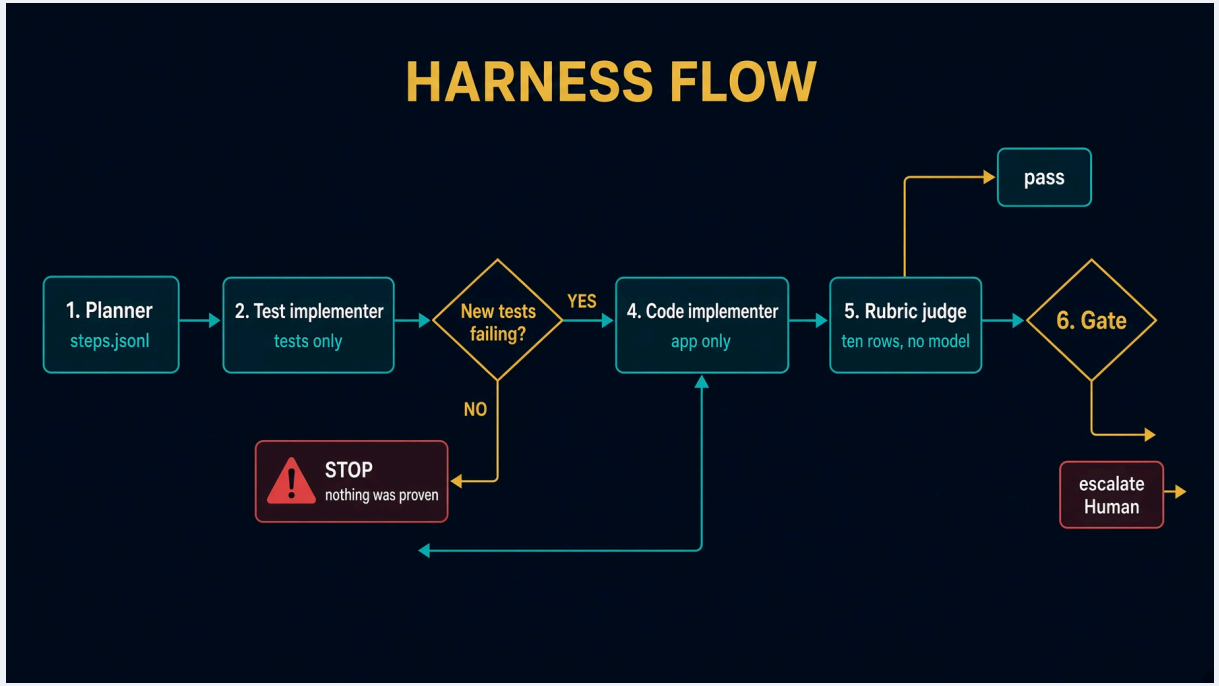
A check that reports success while measuring the wrong thing is worse than no check.

Artifact. A reusable evaluation harness: tests first, prove them red, code until green, judge, gate.

Learning objectives

- Implement `red_gate` so only **new** failing ids count
- Implement `score_attempt` as one call to `rubric.score`
- Implement `run_loop` so Python counts retries, not the model
- Configure two doers with disjoint write scope
- Validate `--doer none` escalates (honesty) and `--doer reference` can pass
- Troubleshoot the stall of computing rubric rows by hand

Starting architecture



You fill three function bodies in `harness.py`. Graph Engineering: each criterion becomes a test step and a code step in `steps.jsonl`.

There is no `loops/` package. Do not recreate it.

Prerequisites

```
cd labs/lab2_implementer
```

Pick one tool and paste its prompt:

```
claude -p "$(cat prompts/claude-code.md)"
# or: codex exec "$(cat prompts/codex.md)"
# or: grok -p "$(cat prompts/grok-build.md)"
# or: opencode run "$(cat prompts/opencode.md)"
```

CRM clone from Lab 1 should already sit at `../../work/northwind-field-crm`.

`.claude/settings.json` denies writes to `loops/`, `scripts/`, and `work/**/tests/**`.

The stub. Three functions. The order is the lesson

```
def red_gate(before: RunResult, after: RunResult) -> set[str]:
    raise NotImplementedError("fill me in")

def score_attempt(contract: Contract, **evidence) -> rubric.Score:
    raise NotImplementedError("fill me in")

def run_loop(contract: Contract, budget: int = 3) -> dict:
    raise NotImplementedError("fill me in")
```

Order, from the module docstring:

```
tests first  ->  prove them red  ->  code until green  ->  judge  ->  gate
```

`red_gate`. New failing ids, not any failing ids

A test that already existed and still fails is not proof of a new contract.
A test that passes before any code exists proves nothing.
An empty result must stop the loop.

Body Rick types (and the Deep Agents port ships):

```
def red_gate(before, after) -> set[str]:
    seen = before.junit.passed_ids | before.junit.failed_ids
    return {tid for tid in after.junit.failed_ids if tid not in seen}
```

The DA core names this `implementer._new_test_ids`.

`score_attempt`. One line. Do not compute rows

"The tests passed" is one row of ten. Absent kwargs become failing rows on purpose.

```
def score_attempt(contract: Contract, **evidence) -> rubric.Score:
    return rubric.score(contract=contract, **evidence)
```

That is the whole function. The common stall is writing ten `if` blocks.

#	ROW	EVIDENCE
1	`tests_ran`	junit exists and is non-empty
2	`tests_passed`	failures+errors == 0
3	`red_first`	`red_ids` non-empty
4	`coverage_floor`	line-rate vs floor (80)
5	`criteria_covered`	every criterion has a passing test
6	`steps_done`	no open plan steps
7	`ui_has_e2e`	UI files changed ⇒ e2e green
8	`format_clean`	format task ok
9	`lint_clean`	lint task ok
10	`write_scope`	no violations (and scope was checked)

`run_loop`. Python holds the retries

```
def run_loop(contract, budget: int = 3, ticket_id: str = "T001", doer: str = "reference") -> dict:
    return implementer.run(
        repo=contract.repo, ticket_id=ticket_id, budget=budget, doer=doer,
    )
```

Saturday this folder has no `implementer.py`. Rick types a body that calls `gates.decide` in a `while` loop, or you watch him fill it. The shipped eight-step loop lives in `solutions/sol2_implementer_deep_agents/implementer.py`.

Pass `doer` through. Hardcoding `reference` makes `--doer none` impossible.

Two doers. Scope is a missing path

From `contract.py` defaults:

```
planner            write_allow: steps.jsonl
test_implementer   write_allow: tests/**
code_implementer   write_allow: app/**, src/**    write_deny: tests/**
judge              write_allow: []                write_deny: **
```

Deny always beats allow. Empty allow permits nothing. `Judge` has no `write` method.

The code implementer cannot weaken a test, not because it was told not to, but because it holds no write path to one.

`gates.decide`. Three exits, no fourth

1. Rubric green (`judge_done is None`) → pass
2. Rubric green and final judge agrees → pass
3. Rubric green, judge says not done → escalate
4. `signature == previous_signature` → escalate (not converging)
5. money budget spent → escalate
6. iteration budget spent → escalate
7. else → retry, `final_attempt` if the next one is last

`signature` is the tuple of **failed row names**, not the wording. Two equal signatures mean the last attempt changed nothing.

Commands. Live first.

Saturday self-check, from `labs/lab2_implementer`:

```
task test    # python3 -c "import harness; print('ok')"
```

Instructor demo, from the Deep Agents driver. Skills are mounted. Python owns the red gate.

```
cd ../../solutions/sol2_implementer_deep_agents
task test-setup
task test
task e2e
task table          # judge writes must print no
task run -- --ticket T001 --doer none
task run -- --ticket T001 --doer reference
```

`--doer none` and `--doer reference` need no key. `--doer deep` needs `task setup` and `ANTHROPIC_API_KEY`.

After class: `.agents/skills/test-ticket-implementer/` plus `HOW_TO_RUN.md` in that folder.

Expected results

`--doer none` writes nothing. The red gate must escalate:

```
gate: escalate
reason: red gate: no new test was observed failing. A test that passes before any code exists proves nothing.
```

If this run were green, the harness would be lying.

`--doer reference` copies `known-good` into `tests/**` then `app/**`, each phase bound by that role's WriteScope. Expect ten PASS rows and `gate: pass`.

Receipt. Three claims or nothing

`scripts/receipt.py` writes `.harness/receipt.json`.

1. Suite passed (exit 0 **and** readable junit **and** no failures)
2. Ran against **this tree** (`tree_hash` of tracked plus untracked)
3. Ran **after** the newest source edit

A zero exit code with no test report is the silent-skip bug wearing a green shirt.

The push-gate refusal you hit live lives on the CRM clone as a Claude Code hook, not under `labs/lab2_implementer/.claude/`. Read the refusal. It is the lesson.

Troubleshooting

SYMPTOM	LIKELY CAUSE	RESOLUTION
Computing ten rows by hand	stall on `score_attempt`	`return rubric.score(contract=contract, **evidence)`
Returning all failed ids	`red_gate` used `after.failed`	subtract `before` ids
Edited `loops/` or `rubric.py`	ignored the prompt	fill only `harness.py`
`task loop:implementer` missing	engine deleted	`task run` from `sol2_implementer_deep_agents`
`--doer none` is green	red gate not stopping	empty new-ids must escalate
`import _root` fails	leftover TROUBLESHOOTING	lab stub does not import it

Fall behind

There is no drop-in `harness.py`. Watch Rick finish. Save first:

```
cp harness.py harness.py.my-attempt
```

See `FALL-BEHIND.md`. Restore with `git checkout -- harness.py`.

Take-home, not Saturday: issues 118. `labs/takehome/deep-agents/loop.py` and `labs/takehome/agent-sdk/loop.py` fill `build` and `run`. Those ports are a different runtime, not a drop-in for this stub.

Recap

What we built. Three functions that make a loop honest: new red ids, a ten-row rubric, Python-owned retries.

Takeaways

1. Two doers. Disjoint scope. The judge has no write method.
2. A test that passes before any code exists proves nothing.
3. "The tests passed" is one row of ten.
4. Same signature twice is stop.
5. A receipt proves three things, or it proves nothing.

Lab 3. Research assistant over MCP

A question in. A cited brief out. Same graph, different object.

25 minutes. Fill `loop.py`. Two functions. Nothing else.

Work from `labs/lab3_research`.

What you will build

Two function bodies:

```
def plan_questions(question: str) -> list[str]: ...
def check_brief(body: str, sources: list[str]) -> brief.BriefScore: ...
```

Already shipped: `brief.py` (the judge). Do not reimplement grounding.

Final outcome. Three checkable sub-questions. Four arithmetic rows on the brief. No model in the judge.

Why this lab exists

A code loop stops when tests go green. A research loop has no equivalent. "Keep searching until confident" is not a stop condition.

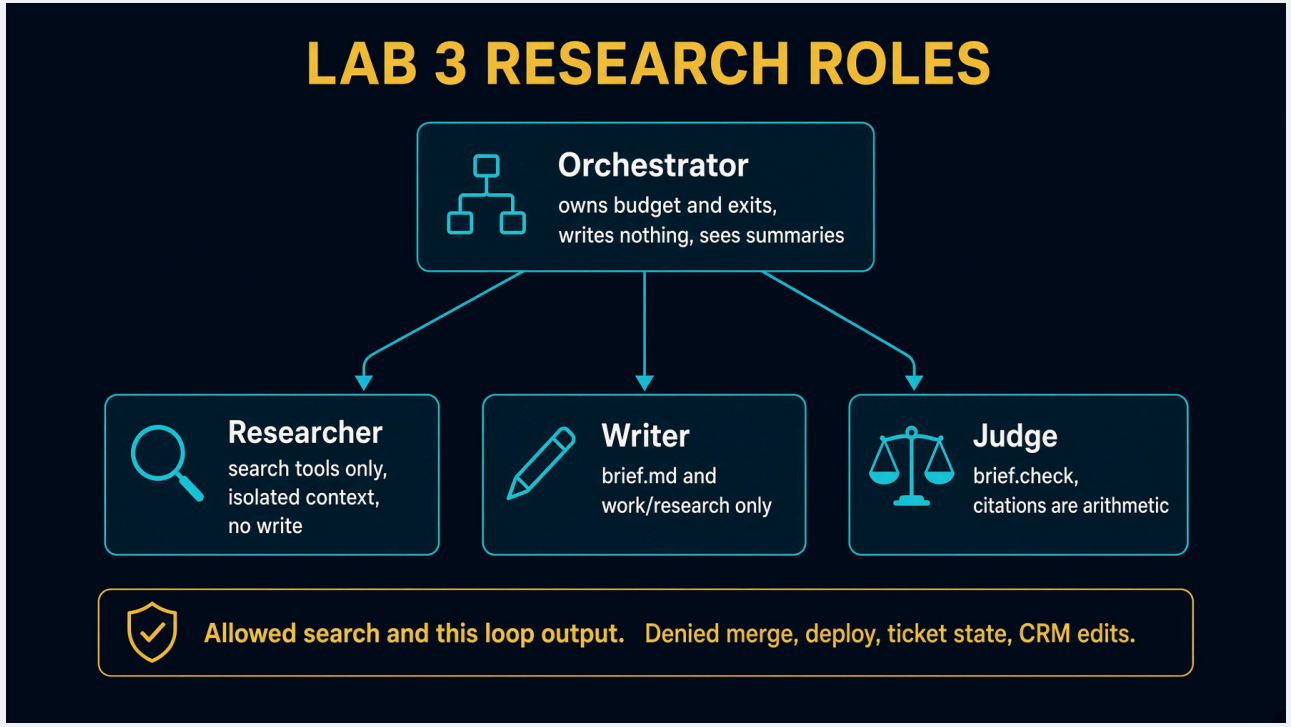
Tool output is untrusted input. Search results are a document the internet wrote, not a system prompt.

Saturday uses the fixture backend. No signup form.

Learning objectives

- Implement `plan_questions` as three checkable sub-questions
- Implement `check_brief` as `return brief.check(body, sources)`
- Configure a tool boundary: search in, merge denied
- Validate `has_sources`, `grounded`, `cited`, `style`
- Troubleshoot the stall of reaching for a model judge

Starting architecture



Allowed: search and this loop's output. Denied: merge, deploy, ticket state, CRM edits.

Prerequisites

```
cd labs/lab3_research
claude -p "$(cat prompts/claude-code.md)"
```

Worth reading: `brief.py`, `MCP.md`.

`.mcp.json` at the repo root ships `context7`. Perplexity is optional. Fixture when the room has no wifi.

`.claude/settings.json` denies writes to `loops/`, `scripts/`, and CRM tests. There is no `loops/` package.

The stub

```
def plan_questions(question: str) -> list[str]:
    raise NotImplementedError("fill me in")

def check_brief(body: str, sources: list[str]) -> brief.BriefScore:
    raise NotImplementedError("fill me in")
```

Fill only `loop.py`. Do not edit `brief.py`. Do not edit `solutions/`.

`plan_questions`. Three strings is enough

A plan step you cannot check is a wish. The common stall is over-designing this function. This is Graph Engineering at lab scale: three checkable nodes, not a wish list.

```
def plan_questions(question: str) -> list[str]:
    return [
        question,
        f"{question} common mistake",
        f"{question} how to verify",
    ]
```

That is the body in `solutions/sol3_research_deep_agents/researcher.py`. Swapping in a model planner is the stretch. Downstream checks do not change.

`check_brief`. Arithmetic. No model

```
def check_brief(body: str, sources: list[str]) -> brief.BriefScore:
    return brief.check(body, sources)
```

ROW	PASS WHEN
`has_sources`	`bool(sources)`
`grounded`	every `[n]` is in `1..len(sources)`
`cited`	every claim paragraph has a `[n]`
`style`	zero em dashes outside code spans

A confident sentence nobody can trace is the failure that matters.

Why style is a row

House style forbids em dashes. A model will argue. Python will not.

`brief.strip_em_dashes` stashes code spans, then comma-heavy lines become `:` , light lines become `;` .

Commands. Live first.

Saturday self-check:

```
task test    # import loop; print('ok')
```

Instructor demo of the cited brief, Deep Agents port:

```
cd ../../solutions/sol3_research_deep_agents
task test
task table
task brief -- --question "sqlalchemy nullable datetime column" --backend fixture
```

The question is boring on purpose. It matches `fixtures/research.json`.

After class the Agent SDK port grows this into a white paper, Arctic Fox PDF, and a secret gist. See `HOW_TO_RUN.md` and `.agents/skills/e2e-test-research-report/`.

Expected fixture result

```
PASS  has_sources    3 sources retrieved
PASS  grounded       every citation resolves
PASS  cited          every paragraph cites a source
PASS  style          0 em dashes

backend: fixture   budget: $0.00 / $0.20 (soft $0.10), 3/8 calls
gate:    pass
```

Empty findings → no body claims → `has_sources` fails → escalate. Never ship an uncited brief.

Budget. Soft warns. Hard raises

Live: `max_usd=0.20`, `max_calls=8`, `soft_usd=0.10`. Perplexity costs 0.006 per call. Fixture costs nothing, which is why Saturday still teaches the cap.

The ninth search does not run. `BudgetExceeded` is a `RuntimeError`, not a nudge.

Four stops: call budget, dollar budget, stable failure, no-source escalates.

Troubleshooting

SYMPTOM	CAUSE	FIX
Reaching for a model in `check_brief`	stall	`return brief.check(...)`
Over-designed planner	stall	three strings
`task loop:research` missing	engine deleted	`task brief` in `sol3_research_deep_agents`
Uncited brief shipped	skipped `has_sources`	empty findings escalate
Em dashes in the brief	model argued	`brief.strip_em_dashes`

Fall behind

```
cp loop.py loop.py.my-attempt
```

Watch Rick type. There is no drop-in. Later: `git checkout -- loop.py`.

Take-home ports are a different runtime, not a drop-in for this stub.

Recap

Same graph as Module 1. New object: a question. New wall: one search tool.

Cost is an architecture problem, not a pricing problem. Do not shop for a cheaper model first.

Takeaways

1. Three sub-questions is a plan you can check.
2. Citations are arithmetic.
3. The orchestrator sees a summary, never the dump.
4. Merge is not a tool this loop holds.
5. Saturday object is a cited brief. Paper pipelines are take-home.

Lab 4. Broken PR fixer, unattended

A failing branch in. A green one out, or an honest explanation of why not.

18 minutes, not 25. Energy is lowest. Keep moving.

Work from `labs/lab4_fixer`.

What you will build

Two functions in `loop.py`:

```
def summarize_failure(run_result: RunResult) -> str: ...
def repair_until_green(contract: Contract, budget: int = 3) -> dict: ...
```

Already shipped: `contract.py` (the repo contract, 344 lines).

Final outcome. A loop that can give up out loud.

Why this lab exists

Nobody is watching. Exits matter more than successes.

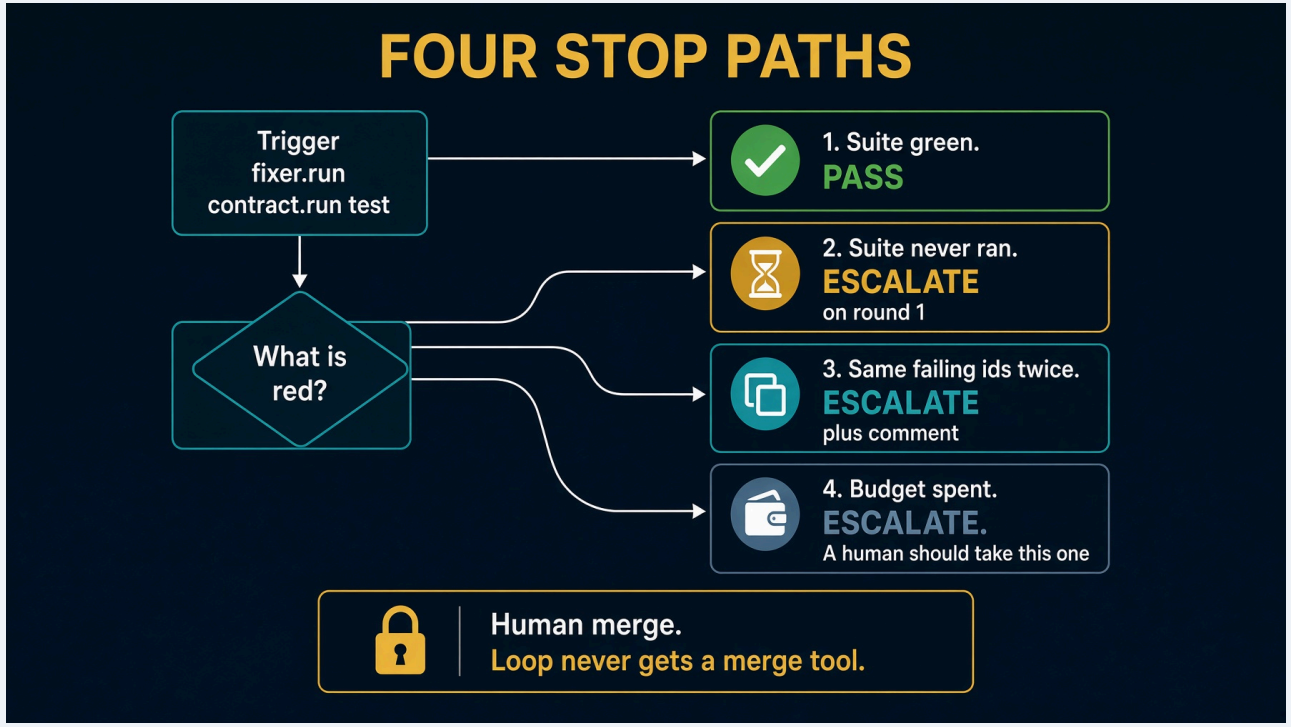
Giving up is allowed. Giving up silently is the bug.

A model that may both act and verify can invent its own evidence. That is why the receipt exists.

Learning objectives

- Implement `summarize_failure` so the error is not lost in the middle of a log
- Implement `repair_until_green` with four stop paths
- Stash Module 2 work **before** `broken-pr` checkout
- Validate `--doer none` leaves "A human should take this one"
- Troubleshoot a loop that wants to delete an earlier lab's work

Starting architecture



Cast is three roles, not five. No planner. The work is defined by what is red.

Stash first. Say it out loud

```
git -C ../../work/northwind-field-crm stash --include-untracked
```

`checkout()` on a dirty tree refuses rather than deleting Lab 2 work:

```
cannot switch to broken-pr: northwind-field-crm still holds
the work from an earlier lab.
  keep it:     git -C ... stash --include-untracked
  discard it:  git -C ... checkout -- . && git -C ... clean -fd
```

That refusal is on brand.

CRM branches

BRANCH	STATE
`main`	no due dates, ~75% coverage, below floor
`known-good`	due dates implemented, everything green
`broken-pr`	dropped the null guard, one test red

`--branch broken-pr` is what makes this real. Point it at a green branch and it reports a pass and proves nothing. Same shape as the red gate.

The stub

```
def summarize_failure(run_result: RunResult) -> str:
    raise NotImplementedError("fill me in")

def repair_until_green(contract: Contract, budget: int = 3) -> dict:
    raise NotImplementedError("fill me in")
```

Fill only `loop.py`.

`summarize_failure`

Sending the whole log puts the failure in the middle. Lost in the Middle, third time today.

```
def summarize_failure(run_result) -> str:
    failed = sorted(run_result.junit.failed_ids)
    lines = [f"{len(failed)} failing: {', '.join(failed[:5])}"] if failed else ["the suite is red"]
    error = ERROR_IN_OUTPUT.search(run_result.output or "")
    if error:
        lines.append(error.group(0).strip()[:200])
    return "\n".join(lines)
```

Four stop paths

1. Suite green → pass
2. Suite never ran → escalate on round 1
3. Same failing ids twice → escalate, leave a comment
4. Budget spent → escalate, "A human should take this one"

Scope violations escalate even if the suite is green. Reaching green by editing the failing test is not a fix.

Commands. Live first.

Saturday: `task test` imports `loop`.

Stash Lab 2 work first. Then demo the live fixer from the Agent SDK port:

```
git -C ../../work/northwind-field-crm stash --include-untracked
cd ../../solutions/sol4_fixer_agent_sdk
task setup
task test
task table          # judge writes must print no
task clone
task reset          # checkout broken-pr. Refuses a dirty clone.
task run -- --branch broken-pr --doer reference
task run -- --branch broken-pr --doer sdk
```

`--doer reference` needs no key. `--doer sdk` needs `ANTHROPIC_API_KEY`. `permission_mode` is `dontAsk`. Merge is never a tool.

Read `HOW_TO_RUN.md` and `E2E_PLAN.md` in that folder. The Deep Agents twin is the graph, not the repair loop.

Expected `--doer none`

```
attempt 1: 1 failing -> retry
  1 failing: tests.test_overdue::test_overdue_ignores_tasks_with_no_due_date
attempt 2: 1 failing -> escalate

gate: escalate
The fixer gave up.
A human should take this one.
```

`--doer reference` against `broken-pr`: `gate: pass`, files copied from `known-good` inside `app/**` only.

MAST, in one slide

Most agent failures are not model failures. 1,642 traces. Cemri et al. arXiv:2503.13657.

CATEGORY	SHARE	THIS HOUR
System design	41.8%	graph, scope, budget, trigger
Inter-agent	36.9%	summaries, not dumps
Verification	21.3%	pytest, receipt

Every one of those three is something you build, not something you buy.

Troubleshooting

SYMPTOM	CAUSE	FIX
Checkout refuses	Lab 2 files dirty	stash, out loud
`--doer none` is green	loop is lying	same ids twice escalate
Edited `tests/**`	scope not enforced	deny list on the coder
Silent give-up	missing comment	"A human should take this one."

Recap

Takeaways

1. Stash first.
2. Giving up is allowed. Giving up silently is the bug.
3. Merge is never a tool.
4. If you cannot read the last score, you cannot debug at 2 a.m.

The loop is the product. The prompt is not.

sol1_enhancer. Claude Code answer

The Saturday plugin, finished. Work from this folder. It is standalone.

Read `HOW_TO_RUN.md` and `DESIGN_DOC.md` before you change a poll.

Saturday Lab 1 fills `.claude/` in `labs/lab1_enhancer`. This folder is that answer.

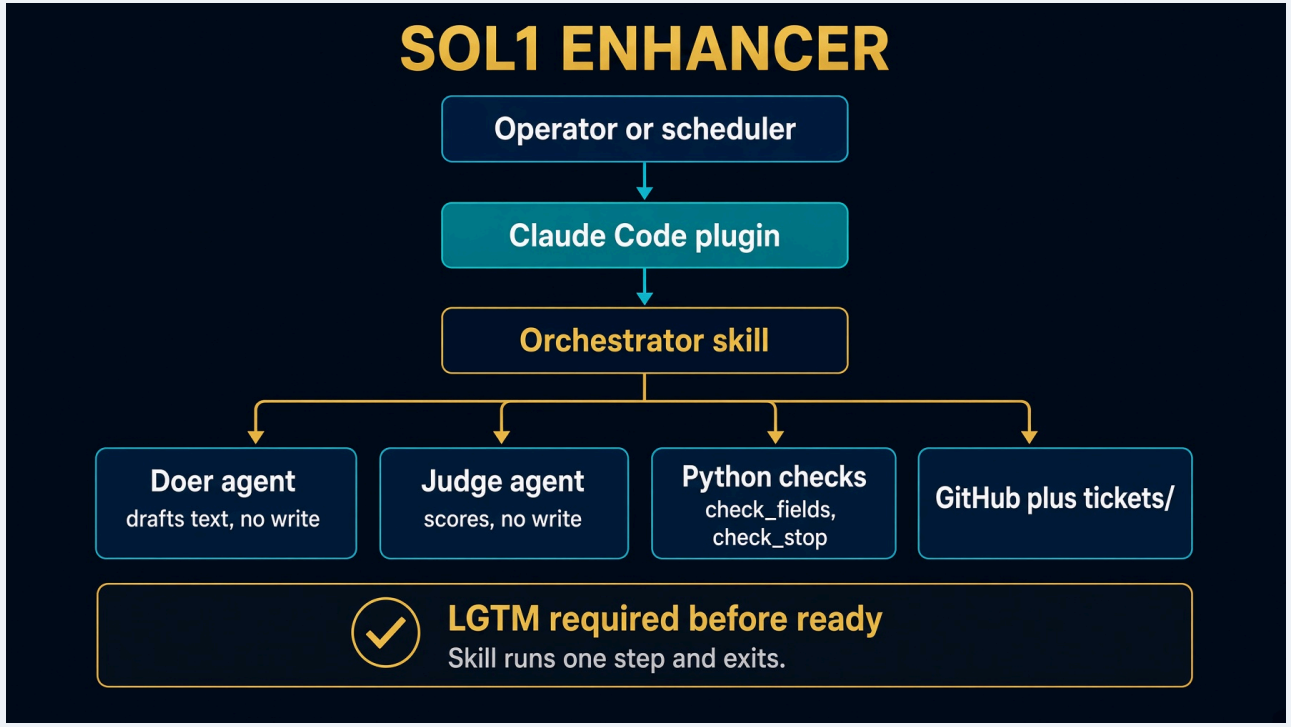
What this folder does

One poll. Every open draft in your fork. Judge reports facts. Python computes ready. Orchestrator alone writes GitHub.

```
cd solutions/sol1_enhancer
cp config.json.example config.json
task clone
task create-test-tickets
task run --
```

`task run` never opens issues. That is `task create-test-tickets`.

Architecture



Doer and judge cannot write the real issue. `LGTM` is required before `ready`.

Setup, once

```
cp config.json.example config.json   # fork_owner
task clone                           # ../../work/northwind-field-crm
task create-test-tickets             # T900 bug, T901 ui, T902 feature, plus T001
```

Paths are built from `TASKFILE_DIR`. You can invoke `task` from anywhere.

One poll. Then LGTM

```
task run --
```

First poll always runs one round. Comments do not start a round. Human comments exact `LGTM` on a green rubric. Next poll sets `state: ready` and `loop: implementer`.

```
task poll-forever --
```

Seminar stand-in. `Ctrl-C` when you are done. Production is Actions. See `GITHUB-ACTIONS.md`.

If it looks hung

`claude -p` prints nothing until the run finishes.

```
"debug": true
```

```
touch debug.log && tail -f debug.log
```

Hooks write one line per tool call. Leave debug false for a normal run.

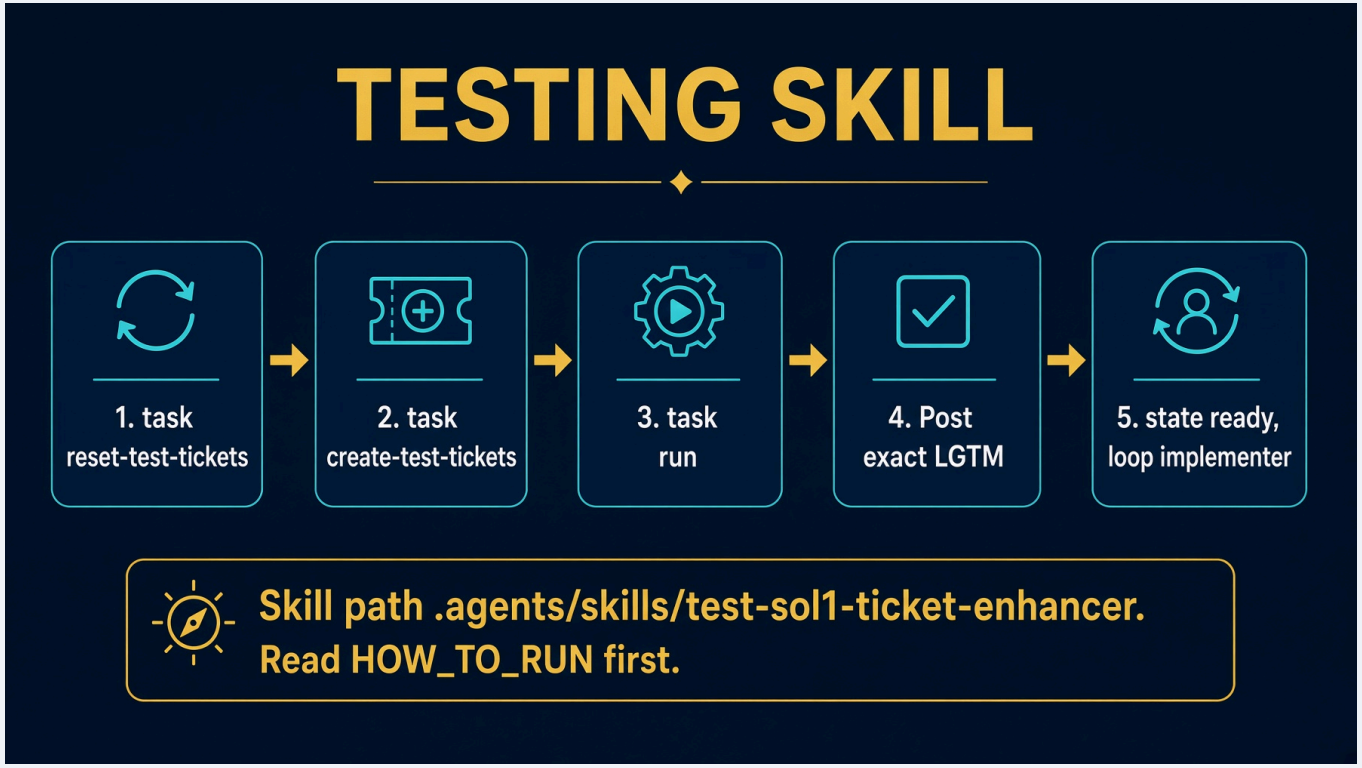
Retest from scratch

Closing an issue by hand is not a reset.

```
task reset-test-tickets
task create-test-tickets
task run --
```

To replay one ticket: keep the issue number, restore draft frontmatter, delete `.harness/last-enhancer-T901.json`, `gh issue reopen`.

Testing skill



`.agents/skills/test-sol1-ticket-enhancer/`

Works on every `solutions/sol1_*` folder. Read that folder's `HOW_TO_RUN.md` first.

Troubleshooting

SYMPTOM	FIX
Skill not found	`cd solutions/sol1_enhancer`
`enhanced` label, body unchanged	rewrite the ticket, then label
`issue N is closed`	`task reset-test-tickets`, not a hand close
no GitHub issue	`task create-test-tickets`
already ready / implementer	reset that ticket if you meant to rerun

Recap

Trigger outside. Exits inside. Judge has no write. Ready is arithmetic.

The loop is the product. The prompt is not.

Spillwave Solutions | spillwave.com

sol1_enhancer_codex

Take-home. Isolation is a process sandbox, not a per-agent tool list.

`bin/role.sh` starts the judge and the doer as their own read-only `codex exec` processes. Do not lose the execute bit.

Read `HOW_TO_RUN.md`, `SPEC.md`, and `IMPLEMENTATION_NOTES.md`.

Setup

```
cd solutions/sol1_enhancer_codex
cp config.json.example config.json
task clone
task fence-check
```

`task fence-check` must **not** report `TRUSTED`. If it does, the sandbox is not real. See `IMPLEMENTATION_NOTES.md`.

Checks with no live poll

```
task test
task checks
python3 .agents/skills/enhancer-loop/scripts/check_fields.py --demo
python3 .agents/skills/enhancer-loop/scripts/check_stop.py --demo
```

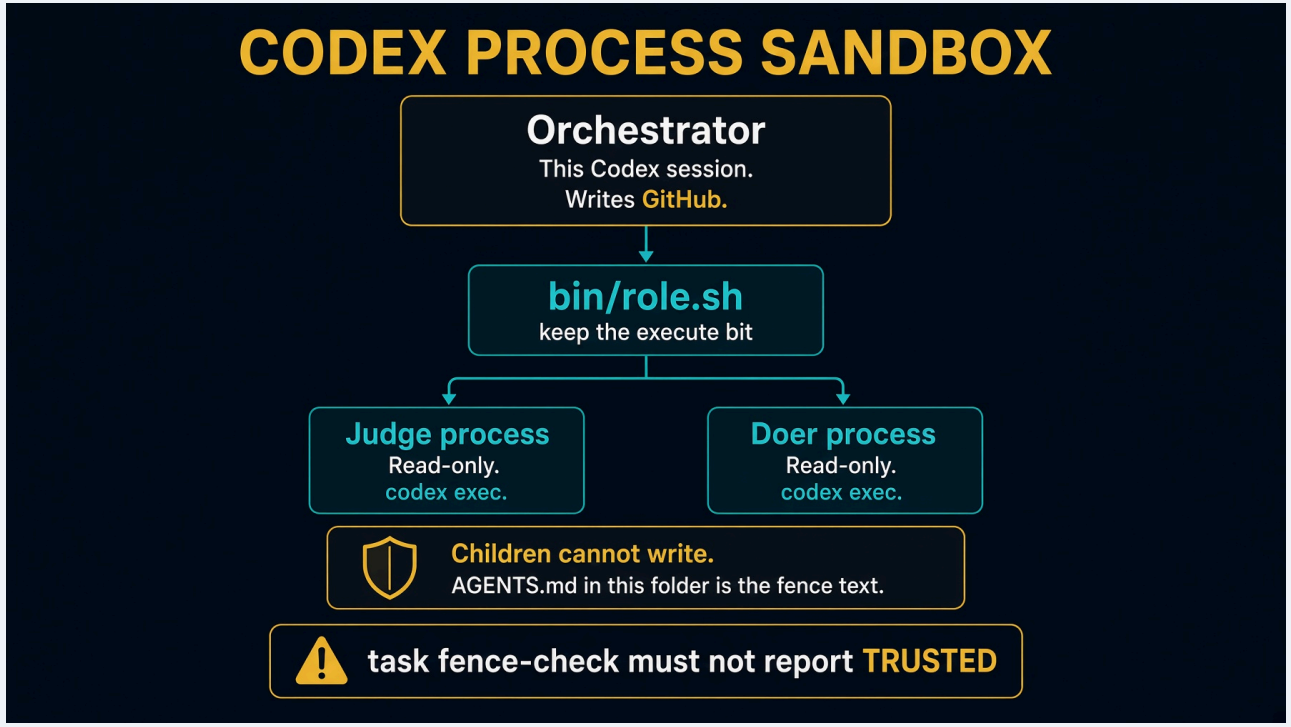
One poll. Budget five minutes

```
task create-test-tickets
timeout 420 task run --
task poll-forever --
```

One poll starts three model processes: judge, doer, judge again. A full round that promotes a candidate runs about four minutes.

A run that produces no output for minutes is usually a hang. `IMPLEMENTATION_NOTES.md` lists the two that look identical from outside.

Architecture



Orchestrator writes GitHub. Children are read-only. `task fence-check` must not report TRUSTED.

Testing skill

`.agents/skills/test-sol1-ticket-enhancer/`

Same reset, seed, poll, `LGTM` ritual. Point `SOLUTION` at this folder. Read `HOW_TO_RUN.md` first.

Troubleshooting

SYMPTOM	FIX
`TRUSTED`	sandbox is not a fence. `IMPLEMENTATION_NOTES.md`
role.sh not found	restore the execute bit
Hang, no output	`timeout 420`. Then the notes.
Closed issue	`task reset-test-tickets`

Recap

Process sandbox. Children cannot write. Orchestrator alone talks to GitHub.

Expect five minutes, not one.

sol1_enhancer_opencode

Take-home. Permission deny on subagents. This port exists. Lab 1 `FALL-BEHIND.md` still says it does not. Ignore that line.

Read `HOW_TO_RUN.md`, `SPEC.md`, and `IMPLEMENTATION_NOTES.md`.

Setup

```
cd solutions/sol1_enhancer_opencode
cp config.json.example config.json
task clone
task create-test-tickets
```

You need `opencode`, `gh`, `jq`, `task`, `python3`. There is no `task test` in this folder. The checks live on the Python scripts.

Checks

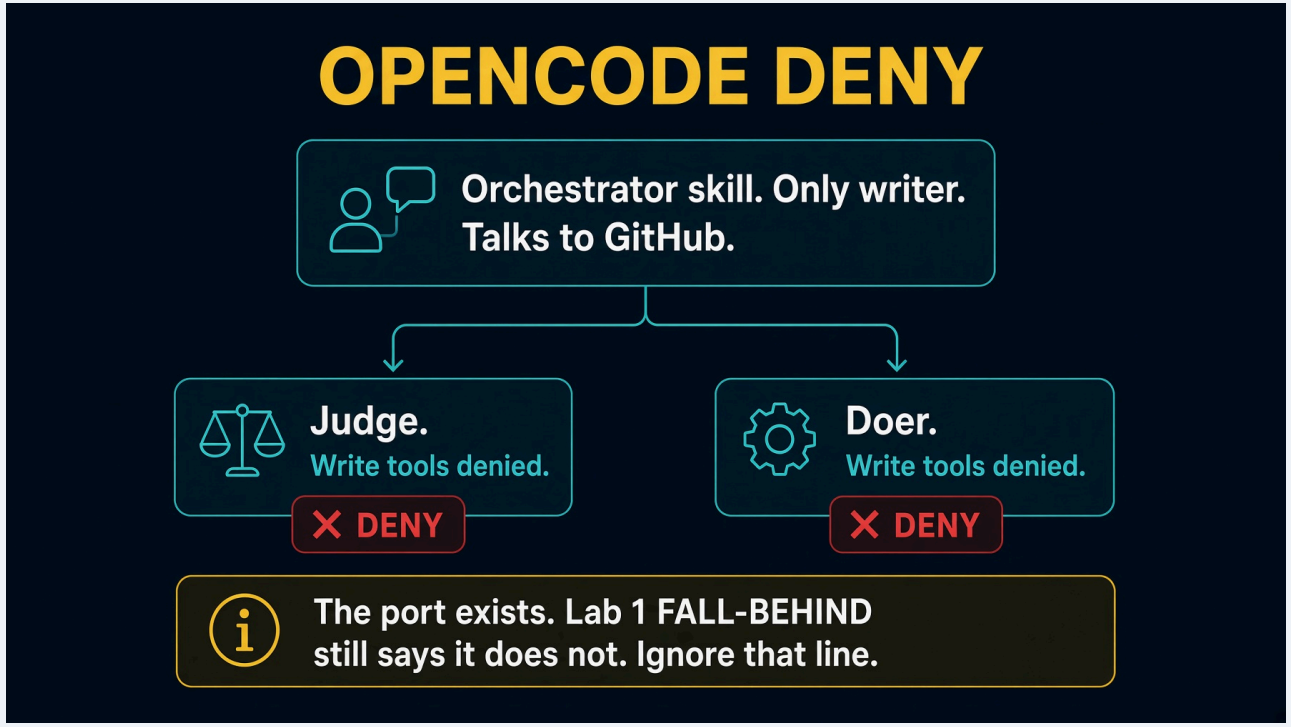
```
python3 .opencode/skills/enhancer-loop/scripts/check_fields.py --demo
python3 .opencode/skills/enhancer-loop/scripts/check_stop.py --demo
```

One poll

```
timeout 360 task run --
task poll-forever --
```

Same poll as Saturday: every open draft, `enhanced` on first rewrite, `LGTM` then `ready`.

Architecture



OpenCode denies write tools on the judge and the doer. Orchestrator is the only writer.

Testing skill

`.agents/skills/test-sol1-ticket-enhancer/`

Point `SOLUTION` at this folder. Read `HOW_TO_RUN.md` first. Skip `task test` here. Run the demo scripts, then the live poll.

Troubleshooting

SYMPTOM	FIX
FALL-BEHIND says no answer	stale. This folder is the answer.
Judge can write	permission deny in OpenCode config
Closed issue	`task reset-test-tickets`

Recap

The port exists. Deny write on the children. Orchestrator writes GitHub.

sol1_enhancer_grok_build

Take-home. Grok only loads a project plugin from a trusted checkout.

Trust is recorded against the git root, so trusting the lab repo covers this folder.

Read `HOW_TO_RUN.md` and `IMPLEMENTATION_NOTES.md`.

Step zero. Trust

```
cd solutions/sol1_enhancer_grok_build
task trust
```

If you see `ticket-enhancer (project, disabled)`, run `grok` here once with no arguments, accept trust, quit, then `task trust` again.

Headless `grok -p` never prompts. Until trust exists, `task run` finds no skill.

Names, not counts

```
grok inspect | grep -E "enhancer-loop|enhancer-judge|enhancer-doer"
```

All three must be listed. The three symlinks under `.grok/skills/` and `.grok/agents/` are what make the loop runnable.

The Plugins line counts directories. `1 agents` can show even when two agent files are loaded. Never read the counts as proof.

Setup and one poll

```
cp config.json.example config.json
task clone
task create-test-tickets
task test
task checks
task run --
task poll-forever --
```

Architecture

Project plugin `ticket-enhancer` plus registration shims. Counts lie. Names do not.

Testing skill

`.agents/skills/test-sol1-ticket-enhancer/`

Point it at this folder after trust is granted.

Troubleshooting

SYMPTOM	FIX
Skill missing	trust the checkout, then the three names
`task run` does nothing	headless never prompts. Trust first.
Symlinks gone after copy	`cp -R` the `.grok` tree. See FALL-BEHIND.

Recap

Trust first. Then the three names. Then one poll.

Counts lie. Names do not.

Spillwave Solutions | spillwave.com

sol1_enhancer_agent_sdk

Take-home. Python owns the loop. The model drafts and grades. It does not write files. It does not run `/enhancer-loop`.

Saturday live path is `solutions/sol1_enhancer`. Do not copy these fences into that folder.

Read `HOW_TO_RUN.md` and `DESIGN_DOC.md`.

Setup. Folder-local venv

```
cd solutions/sol1_enhancer_agent_sdk
cp config.json.example config.json
echo 'ANTHROPIC_API_KEY=sk-ant-...' >> ../../.env
task setup          # .venv + claude-agent-sdk. PEP 668.
task clone
task create-test-tickets
```

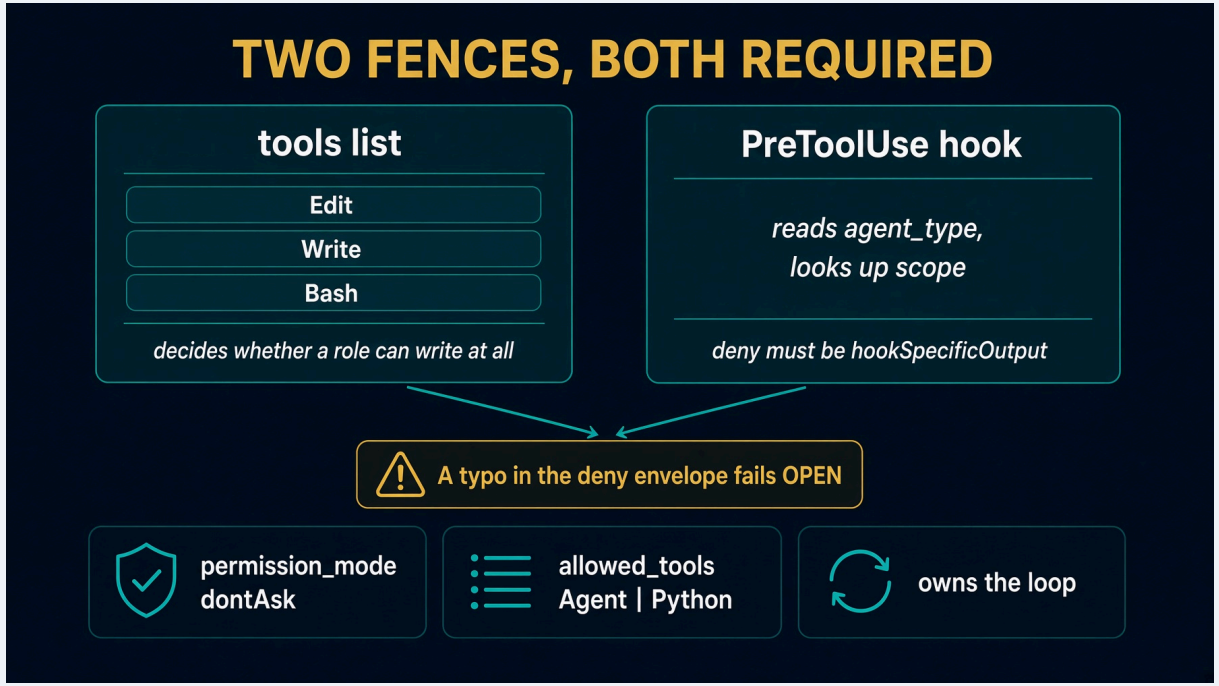
Do not `pip install -r ../../requirements-takehome.txt`. Homebrew Python will refuse.

Scripts with no model

```
task table          # judge writes must print no
task checks
task test           # pytest. No key, no clone, no SDK.
```

If judge prints `yes`, stop. The port is wrong.

Architecture



Python writes the candidate and runs the checks. A typo in the deny envelope fails **open**.

See `docs/diagrams/architecture.svg`.

One poll

```
timeout 420 task run --
task run -- --quiet
task poll-forever --
```

A first poll is three model calls: judge, doer, judge again. Cap it while you develop.

Hung queries dump to `.harness/last-doer-T<id>.md`. Cap is 180 seconds.

Scope. Two places, both required

1. `tools=[...]`. `NO_WRITE` strips Edit, Write, Bash.
2. `PreToolUse` hook `roles.scope_hook`. Empty `{}` = allow. Deny must be `hookSpecificOutput` with `permissionDecision: deny`.

Parent session: `allowed_tools=["Agent"]`. `permission_mode` is not chatting. Python already owns the loop.

Testing skill. Same ritual as Claude Code

`.agents/skills/test-sol1-ticket-enhancer/`

```
task reset-test-tickets
task create-test-tickets
task run --
```

Review live issues. Post exact `LGTM`. Next poll: `state: ready`, `loop: implementer`.

Troubleshooting

SYMPTOM	FIX
Judge writes `yes`	strip Write with `NO_WRITE`
`externally-managed-environment`	`task setup`, not system pip
Hang, no output	read `.harness/last-doer-T<id>.md`
Closed issue	`task reset-test-tickets`

Recap

Python holds the loop. The model only drafts and grades.

`task setup`, `task table`, `task test`, `task run`. Read `HOW_TO_RUN.md`.

sol1_enhancer_deep_agents

Take-home. Deep Agents. Python is the harness. The model drafts and grades.

Saturday live path is `solutions/sol1_enhancer`. Do not copy these fences into that folder.

Read `HOW_TO_RUN.md` and `DESIGN_DOC.md`. Skills are mounted, not pasted.

Setup

```
cd solutions/sol1_enhancer_deep_agents
cp config.json.example config.json
echo 'ANTHROPIC_API_KEY=sk-ant-...' >> ../../.env
task setup          # .venv + deepagents>=0.7
task clone
task create-test-tickets
```

Scripts with no model

```
task table          # judge writes must print no
task checks
task test
```

If judge prints `yes`, stop.

Three fences

Layer 3 is the one people skip. Leaving the default subagent on is how a scoped agent writes anywhere.

See `docs/diagrams/architecture.svg`.

One poll

```
timeout 420 task run --
task run -- --ticket T001
task run -- --ticket T001 --simulate-comment LGTM
task poll-forever --
```

`--simulate-comment` needs `--ticket`. First poll is three model calls.

`task run` is `python3 loop.py --once --repo <target>`. Extra flags after `--` go to `loop.py`.

Skills, not a stuffed prompt

```
skills/doer/SKILL.md
skills/judge/SKILL.md
```

Deep Agents loads the body when the role is invoked. Do not paste `SKILL.md` into a subagent prompt.

Testing skill

`.agents/skills/test-sol1-ticket-enhancer/`

```
task reset-test-tickets
task create-test-tickets
task run --
```

Same GitHub + `LGTM` ritual as the Claude Code answer.

Troubleshooting

SYMPTOM	FIX
Judge writes `yes`	no write tool on the judge
Agent writes anywhere	turn the general-purpose subagent off
Skill not loading	do not shadow `read_file` with a custom tool
No key	`task table` and `task test` still run

Recap

Three fences. Skills mounted. Python holds the loop.

Saturday still lives in `solutions/sol1_enhancer`.

sol2_implementer_deep_agents

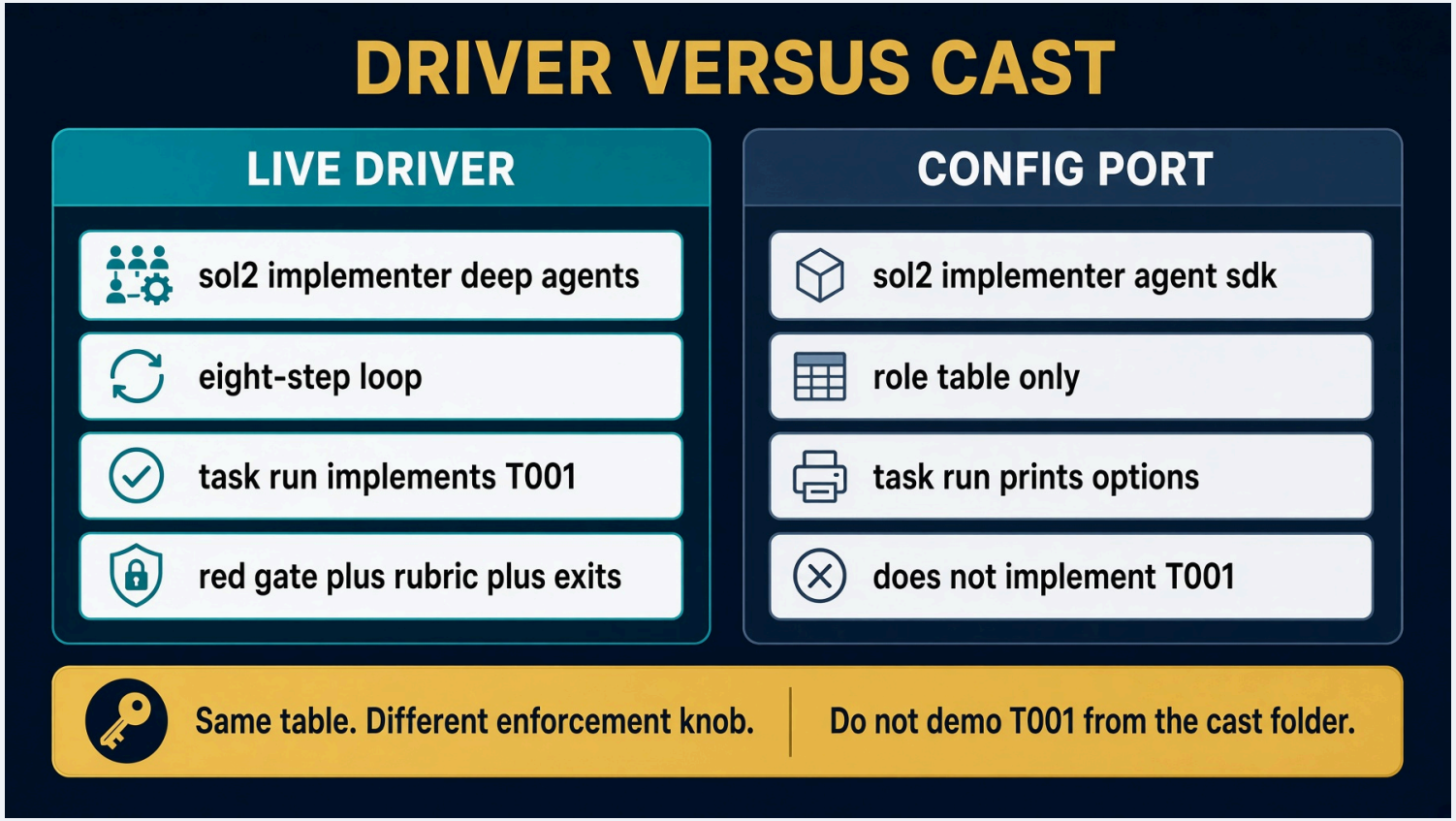
The Lab 2 take-home driver. A ready ticket in. A green rubric out.

Python holds Pass, Retry, and Escalate. Deep Agents is the maker. The red gate is `junit.xml`.

Saturday fills three stubs in `labs/lab2_implementer`. Demo T001 from here.

Read `HOW_TO_RUN.md` and `DESIGN_DOC.md`. Skills are mounted, not pasted.

Driver, not a cast



Demo T001 from this folder. The Agent SDK twin prints options. It does not implement the ticket.

What this folder is

FILE	ROLE
`harness.py`	CLI, `red_gate`, `run_loop`
`implementer.py`	eight-step loop
`doers.py`	`none` / `reference` / `cli` / Deep Agents
`rubric.py` / `gates.py` / `contract.py`	copies, not a library
`roles.py`	`create_deep_agent`
`skills/`	mounted per writing role

`task loop:implementer` is gone from the root Taskfile. Run `task run` here.

Setup. Two venvs on purpose

```
cd solutions/sol2_implementer_deep_agents
cp config.json.example config.json
echo 'ANTHROPIC_API_KEY=sk-ant-...' >> ../../.env
task test-setup     # .venv + pytest. Enough for none and reference.
task clone
```

`--doer deep` needs one more step:

```
task setup          # same .venv, plus deepagents>=0.7
```

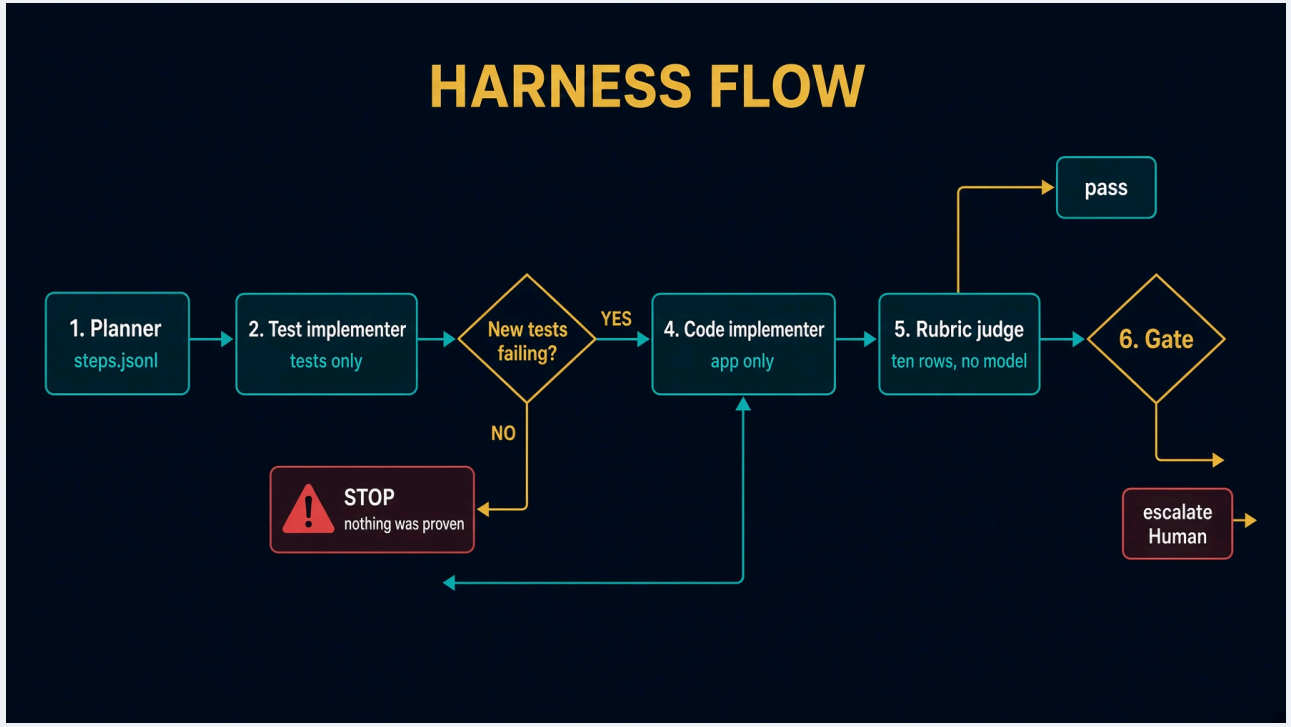
Do not activate the venv. Task uses it.

Scripts with no model

```
task table          # judge writes must print no
task test
task e2e            # offline loop against a disposable fixture
```

If judge prints `yes`, stop. None of these need a key, Deep Agents, or the public CRM.

Architecture



Python runs the suite through `Contract`. No role holds a shell. See `docs/diagrams/architecture.svg`.

Eight steps in `implementer.run`

1. Read the ticket. Refuse if it is still a draft.
2. `plan_for`: one test step and one code step per criterion. Derived, not generated. Graph Engineering.
3. `test_implementer` writes under `tests/**`.
4. Red gate. Empty new-ids → escalate. Stop. Do not write app code.
5. `code_implementer` writes `app/**` until green. Denied `tests/**`.
6. Ten-row rubric. No model.
7. `judge_done=None`. A green rubric is enough on this path. Session 2 still teaches a model judge.
8. `gates.decide`. Trace to `.harness/last-implementer.json`.

`create_deep_agent` does not count retries.

Red gate

```
def _new_test_ids(before: set[str], after_failed: set[str]) -> set[str]:
    return {test_id for test_id in after_failed if test_id not in before}
```

A test that already existed and still fails is not proof of a new contract.

`--doer none` hits this every time. If that run were green, the harness would be lying.

Three doers

SPEC	NEEDS	BEHAVIOR
`none`	nothing	writes nothing. Honesty check.
`reference`	clone	copies `known-good` inside WriteScope
`deep`	`task setup` + key	Deep Agents makers

```
task run -- --ticket T001 --doer none
task run -- --ticket T001 --doer reference
task run -- --ticket T001 --doer deep
```

`task run` is `harness.py --repo <target>`. Extra flags after `--` go to `harness.py`.

Expected results

`--doer none`:

```
gate: escalate
reason: red gate: no new test was observed failing.
```

`--doer reference`: copies `known-good` into `tests/**` then `app/**`, each phase bound by that role's WriteScope. Ten PASS rows. `gate: pass`.

Skills, not a stuffed prompt

Each writing role mounts `/skills/<role>/`. Deep Agents loads the body when the role is invoked.

`/memory/` routes at `memory/`, not this folder.

Do not paste `SKILL.md` into a subagent prompt. Do not shadow `read_file` with a custom tool.

Testing skill

`.agents/skills/test-ticket-implementer/`

Works on every `solutions/sol2_implementer_*` folder. Track A is this driver. Track B plugs another runtime into it.

Read `HOW_TO_RUN.md` first. Run `task test` and `task e2e` before any live spend.

The product is `.harness/last-implementer.json`, not a process exit code.

Troubleshooting

SYMPTOM	FIX
`task loop:implementer` missing	`task run` in this folder
`--doer none` is green	empty new-ids must escalate
Wrote `tests/**` as coder	scoped write tool, sentence refusal
`--doer deep` refuses	`task setup` first
`from loops` in a file	`test_standalone.py` fails the build
Skill not loading	do not shadow `read_file`

Recap

Python owns the red gate and the three exits. Deep Agents writes tests, then code.

Same table as Saturday. Enforcement is a missing tool, plus a path check, plus Python on the outside.

If a port imports `loops`, the design leaked.

sol2_implementer_agent_sdk

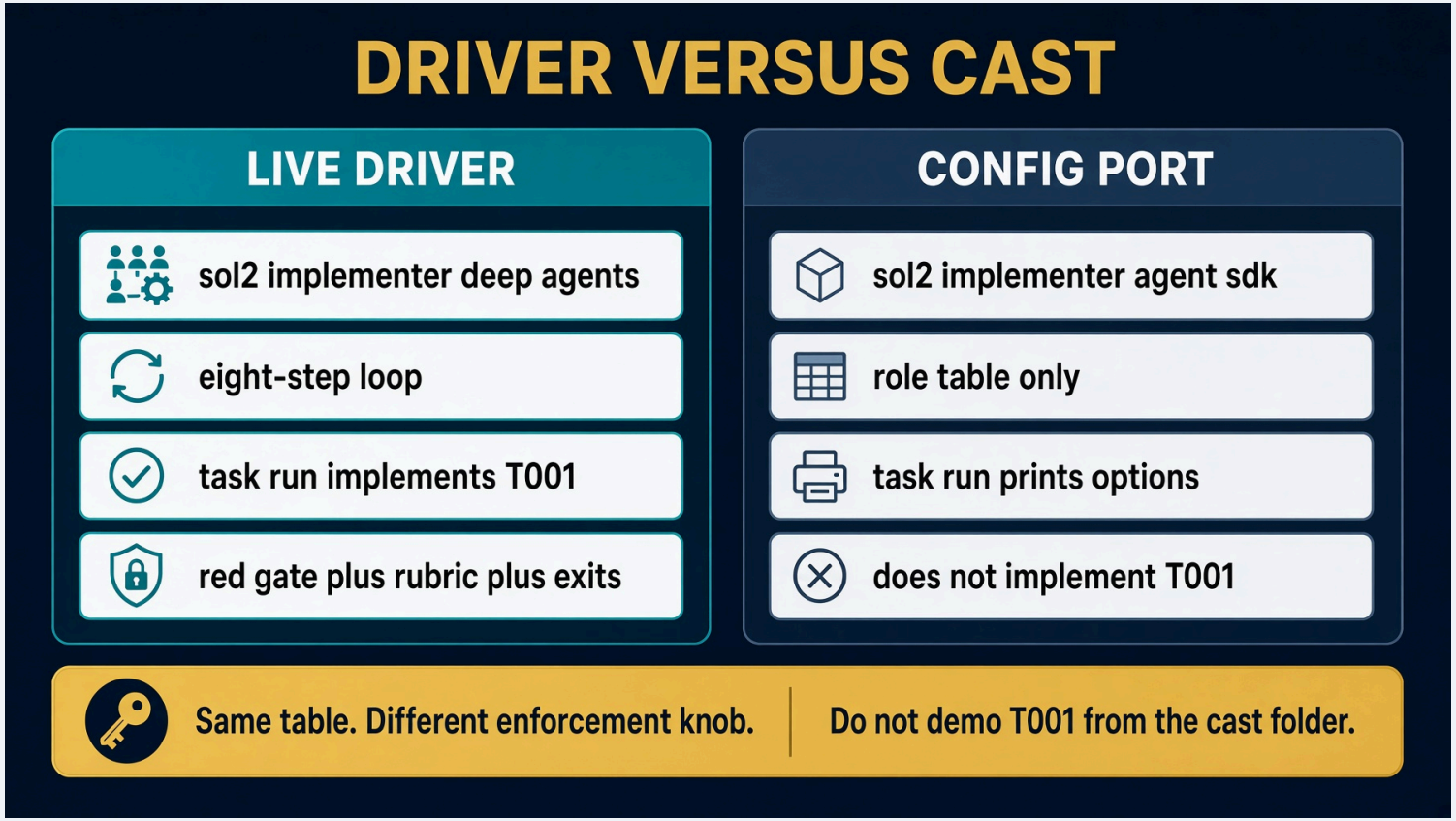
Take-home **configuration port**. It does not run the eight-step loop.

This is the role graph. The live harness is `sol2_implementer_deep_agents`.

`task run` prints `ClaudeAgentOptions`. It does not implement T001.

Read `HOW_TO_RUN.md`, `DESIGN_DOC.md`, `TEST_PLAN.md`, and `E2E_PLAN.md`.

Cast, not the driver



`task run` prints `ClaudeAgentOptions`. It does not implement T001.

What it proves

The role table survived the runtime swap.

```
role                writes  scope
orchestrator        no      nothing
planner             yes     steps.jsonl
test_implementer    yes     tests/**
code_implementer    yes     app/**, src/**   denied tests/**
judge               no      nothing
```

Missing on purpose: `implementer.py`, `gates.py`, `rubric.py`, `doers.py`.

Do not copy this folder into `labs/lab2_implementer/harness.py`. Saturday wants `red_gate` / `score_attempt` / `run_loop`.

Setup. Folder-local venv

```
cd solutions/sol2_implementer_agent_sdk
echo 'ANTHROPIC_API_KEY=sk-ant-...' >> ../../.env
task setup          # .venv + claude-agent-sdk. PEP 668.
task clone          # only if you want live options against a real repo
```

Do not `pip install` into Homebrew Python.

Scripts with no model

```
task table          # judge writes must print no
task test           # pytest. No key, no SDK, no clone.
```

If judge prints `yes`, stop. The port is wrong.

Architecture

`task run --` prints options with `dontAsk` and one PreToolUse hook. No call to `implementer.run`.

Scope. Two places, both required

`tools=[...]` decides whether a role can write at all.

One `PreToolUse` hook reads `agent_type` and looks up that role's scope.

A write with no agent is denied. The parent has no business writing anything.

Deny envelope:

```
hookSpecificOutput.hookEventName = PreToolUse
hookSpecificOutput.permissionDecision = deny
```

A typo fails **open**. The field is `maxTurns`, camelCase.

`task run` is configuration

```
task run --
```

Needs `task setup`. Prints options. Stops.

That is not a ticket run. A test that expects `task run -- --ticket T001` to implement a ticket is testing a product this folder refused to be.

Live T001. Glue, not a second loop

`e2e_t001.py` hands `AgentSdkBackend` to the Deep Agents driver.

Both folders ship `adapter.py` and `roles.py`. Putting both on `sys.path` shadows one of them. The glue loads each folder in an isolated import scope.

Do not invent a shared `loops/` package. Do not copy `implementer.py` into this folder.

Read `E2E_PLAN.md` before spending a token.

Testing skill

`.agents/skills/test-ticket-implementer/`

Track A is `sol2_implementer_deep_agents`. Track B is this backend plugged into that driver.

Run `task test` here first. Do not spend a token to diagnose a failing offline suite.

Plugin files vs Python

```
plugin/agents/...
plugin/skills/...
```

The plugin is the readable specification. Python owns the options. When an agent markdown file and the role table disagree, `options_for` raises.

Troubleshooting

SYMPTOM	FIX
Thought this ran T001	config port. Use Deep Agents `task run`
Judge writes `yes`	strip Write from judge tools
Fail-open writes	full `hookSpecificOutput` deny
`externally-managed-environment`	`task setup`, not system pip
Copied into lab2 stub	Saturday wants three functions

Recap

Config port, not a filled loop. Same table, different enforcement knob.

The eight steps live in `sol2_implementer_deep_agents`. This folder proves the table survived.

`task setup`, `task table`, `task test`, `task run`. Read `HOW_TO_RUN.md`.

sol3_research_deep_agents. White paper

A question in, a cited brief out. A topic in, an evidence-backed white paper out.

LangChain Deep Agents. Two entry points. One role table.

This folder has no `HOW_TO_RUN.md`. Read `SPEC.md`, `DESIGN_DOC.md`, and `Taskfile.yml`.

Saturday Lab 3 still fills two functions. This folder writes `whitepaper.md`.

Two entry points. Same folder.

```
task brief -- --question "sqlalchemy nullable datetime column" --backend fixture
task paper          # nine stages, fixture research
task live           # backend auto. Needs task setup.
```

The brief is the small version of the paper. Both plan questions, search through one tool boundary, and refuse to ship anything uncited.

Setup

```
cd solutions/sol3_research_deep_agents
echo 'ANTHROPIC_API_KEY=sk-ant-...' >> ../../.env
task setup
```

`.venv` plus Deep Agents. Also clones `imagen-diagrams` v0.2.0 and `image-gen` v2.1.0 into `.cache/`.

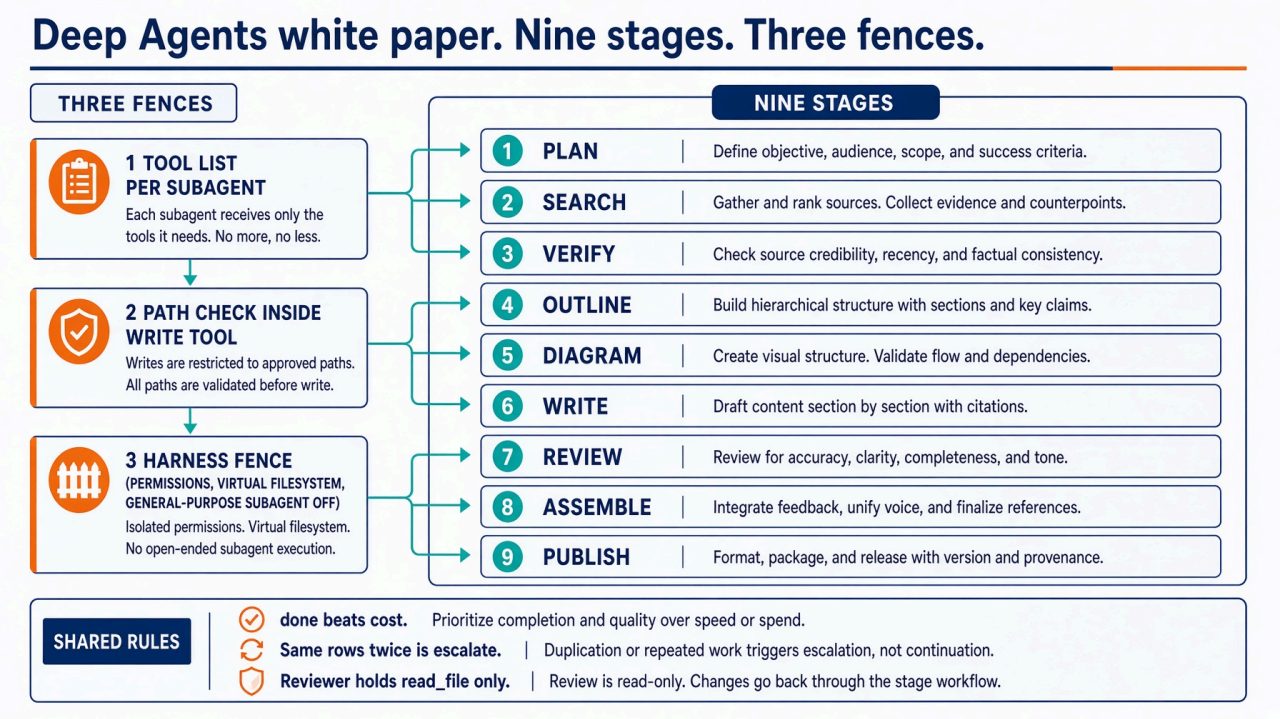
`task test`, `task table`, `task checks`, and `task brief` need none of that.

Scripts with no model

```
task table          # paper cast. Reviewer writes must print no
task checks         # evidence, paper_check, diagrams, publish, mcp_tools
task test
```

If the reviewer prints `yes`, stop.

Starting architecture



See also `docs/diagrams/architecture.svg`.

The paper cast

```
role           writes  scope
orchestrator   no      nothing
planner        yes     plan.json
researcher     no      nothing
verifier       yes     evidence/**     denied: paper/**
diagrammer     yes     diagrams/*.mmd, *.puml
writer         yes     brief.md, paper/**, work/research/**
reviewer       no      nothing
```

A role that would hold the same tools as its neighbour is not a role. It is a prompt, and it belongs in a skill.

Three fences. People skip the third.

Do not shadow `read_file`. That is how `/skills/` goes silent.

Nine stages. Two of them are Python.

#	STAGE	MODEL?	GATE
1	plan	yes	3 to 8 questions, each with a check
2	search	yes	every claim names a source that resolves
3	verify	yes	every important claim has a decided truth state
4	outline	yes	every body section names claim ids that exist
5	diagram	yes	at most 12 nodes, every figure has alt text
6	write	yes	a section cites only the numbers its claims support
7	review	yes	every rubric row passes
8	assemble	**no**	every hard gate in `paper_check`
9	publish	**no**	the gates passed. Opt-in only

Three exits. Done first.

Checked before every stage:

EXIT	WHEN
`done`	stage 8 finished and every hard gate is green
`cost`	the money budget is spent
`max turns`	a stage exhausted its retries

Done beats a spent budget. A run that finished and then noticed it was over its cap did finish.

Cost is checked before every model call, not between stages. A spent budget is never a retry.

Saturday brief is still here

```
def plan_questions(question: str) -> list[str]:
    return [question, f"{question} common mistake", f"{question} how to verify"]

def check_brief(body, sources):
    return brief.check(body, sources)
```

Four arithmetic rows: `has_sources`, `grounded`, `cited`, `style`.

```
task brief -- --question "sqlalchemy nullable datetime column"
```

That is the Lab 3 answer. The paper is the take-home grown from it.

Figures fail closed

Mermaid and PlantUML are source. They are not publication formats.

`imagen-diagrams` v0.2.0 renders `*_imagen.png` and judges it. `image-gen` v2.1.0 is cover art only.

A missing backend writes `<stem>_imagen.prompt.txt` and exits 2. The paper never substitutes SVG.

Publish. Four load-bearing rules.

1. **One gist per paper, forever.** Id in `gist-ids.tsv`.
2. **Secret, never public.** Unlisted. The URL is the credential.
3. **Figures inline.** A gist is flat.
4. **The gate comes first.** `push` refuses when `gates.json` is red.

```
task paper          # never publishes
task publish -- --slug ... --dry-run --out /tmp/staged
```

Testing skill

`.agents/skills/e2e-test-research-report/`

Default lane is the Agent SDK twin. Point it at this folder only when you name it and the tasks exist.

Read `SPEC.md` and `Taskfile.yml` first. `task test` and `task checks` before any live spend.

Troubleshooting

SYMPTOM	FIX
Reviewer writes `yes`	`read_file` only
Agent writes anywhere	turn the general-purpose subagent off
Traceback retry storm	return a refusal sentence
New gist every run	one id per slug in `gist-ids.tsv`
Published a red paper	`push` must refuse
Saturday brief missing	`--question`, not `--paper`
No `HOW_TO_RUN.md`	use `SPEC.md` + `Taskfile.yml`

Recap

A brief for Saturday. A paper for take-home. Same exits.

1. Two entry points. One table.
2. Three fences. The third is the one people skip.
3. Assemble and publish are Python.
4. Done beats cost.
5. Claims outlive the paper.

A role that holds the same tools as its neighbour is a prompt.

sol3_research_agent_sdk. White paper

A topic in. An evidence-backed technical white paper out.

Claude Agent SDK. Seven roles. Python owns the phases.

This is not the old config-only port. Saturday Lab 3 still fills two functions. This folder writes `paper.md`.

Read `HOW_TO_RUN.md` and `DESIGN_DOC.md`.

Setup. Folder-local venv

```
cd solutions/sol3_research_agent_sdk
echo 'ANTHROPIC_API_KEY=sk-ant-...' >> ../../.env
task setup
```

`.venv` plus the Agent SDK. Also pins `imagen-diagrams` v0.2.0 and `image-gen` v2.1.0 in `.cache/`.

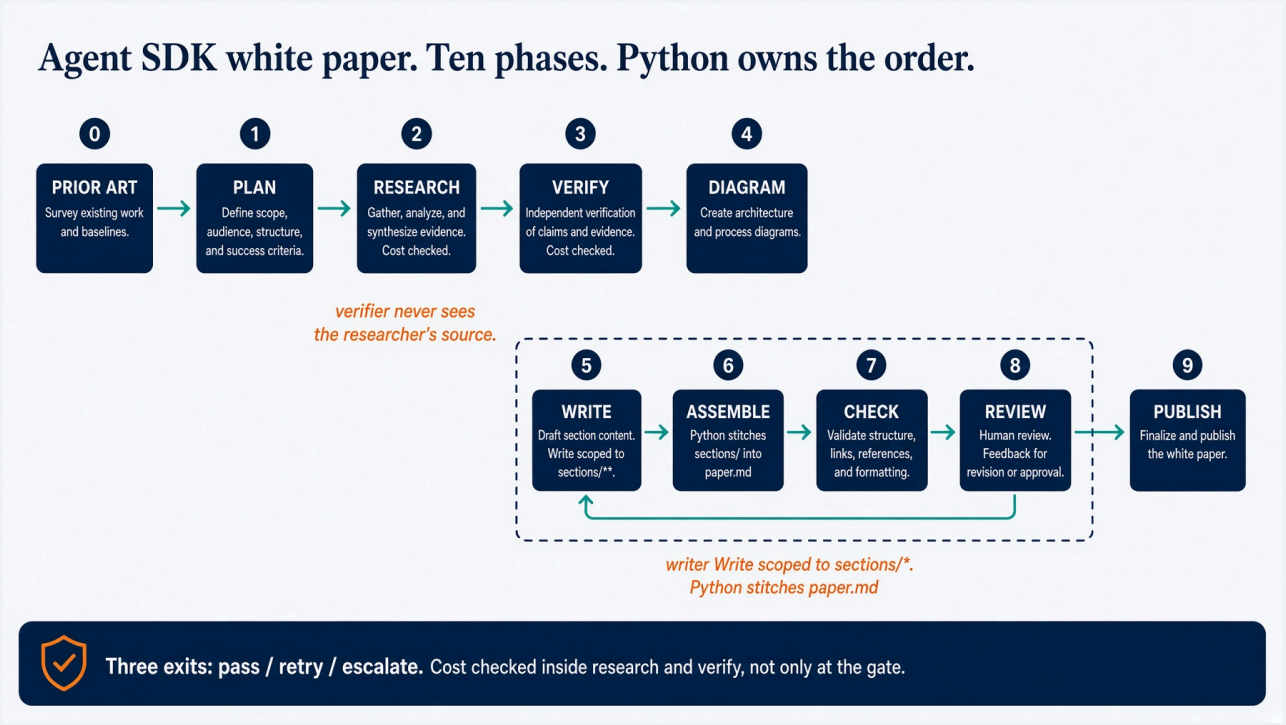
Do not install into Homebrew Python. Do not discover user or parent-project skills. The allowlist is those two plugins only.

Scripts with no model

```
task table          # one writer. Everyone else prints no
task checks
task test
task demo           # recorded fixture. No key, no network.
```

If judge, researcher, or verifier prints `yes`, stop.

Starting architecture



See also `docs/diagrams/architecture.svg`.

The cast. Seven roles, one writer.

```
orchestrator   Task                writes nothing
planner        Read, Glob, Grep    returns plan JSON. Python writes plan.json
researcher     Read + search MCP   writes nothing
verifier       Read + search MCP   writes nothing. Never sees the source.
diagrammer     Read, Glob, Grep    returns diagram source. Python renders.
writer         Read + Write        sections/** only. Cannot reach paper.md
judge          Read, Glob, Grep    writes nothing
```

Two places enforce scope. You need both.

`tools=[...]` decides whether a role can write at all.

One `PreToolUse` hook decides which paths. It reads `agent_type`.

A write with no `agent_type` is denied. So is a write from anyone but the writer. So is a write to `paper.md` or `claims.json`.

sol1 registered one hook per writing role. Several hooks on `Write` all run. An empty dict means no opinion. The first role that shrugs lets another role's write through.

`allowed_tools` is not the parent's list

It is a session-wide permission allowlist. It gates subagents too.

First live run: `allowed_tools=["Agent"]` with `dontAsk`. Every search came back denied.

The researcher did **not** answer from memory. It reported `NO RESEARCH WAS PERFORMED` and returned an empty claim list. The run escalated.

`options_for` now allows the union of what the cast holds. `Bash` is denied at the top.

MCP is declared in Python

Context7 is always configured. Perplexity is included only when `PERPLEXITY_API_KEY` is set.

`.mcp.json` holds a key, so it is gitignored. A fresh clone has no such file. Declare servers in Python or a missing file silently cannot search.

This port has no OpenAI or Bing fallback. A missing root `.mcp.json` must not change the tool boundary.

Ten phases. Python owns the order.

```
0 prior_art   skip if missing              -> prior-art.md
1 plan        sections and questions       -> plan.json
2 research    claims from primary sources  -> sources.json, claims.json
3 verify      independent second look      -> verdicts.json
4 diagram     source in, Python renders    -> diagrams.json
5 write       one section at a time        -> sections/*.md
6 assemble    stitch + references          -> paper.md
7 check       seven deterministic rows     -> check.json
8 review      judge on what a script cannot -> review.json
9 publish     secret gist, on request      -> gist.json
```

Phases 0 to 4 run once. 5 to 8 are the retry cycle.

Cost is checked inside the phases

`--max-usd` default 5.00. Checked inside research, inside verification, and at the gate.

Checking only at the gate is a bug this port had. A twenty-four question research phase can spend the whole budget several times over before the gate ever sees it.

Running out mid-verification leaves remaining claims `unverified`. Marking them verified because the money ran out is the lie.

The check that matters most: `sourced`

A web search cannot refute a citation that was never published. Asking a model whether a reference is real gets you a confident yes.

The only thing that catches a fabricated arXiv id or DOI is looking for it in the text that was actually retrieved.

`checks.ungrounded_identifiers`.

White-paper acceptance runs

```
task e2e-fixture
LIVE_E2E_MAX_USD=10 task e2e-live
REPORT_DIR=work/e2e-loop-engineering-live task pdf
REPORT_DIR=work/e2e-loop-engineering-live task publish-report
```

Fixture uses recorded research plus real `imagen-diagrams` figures. Live needs both keys and never publishes a gist by itself.

PDF uses Arctic Fox. `publish-report` is a secret gist. The URL is the credential.

`task clean` deletes `work/`. The `.cache/` plugins stay.

Testing skill

`.agents/skills/e2e-test-research-report/`

Default folder is this one.

```
task setup && task test && task checks
LIVE_E2E_MAX_USD=10 task e2e-live
```

Use the fixture lane when credentials are absent. Do not describe a fixture lane as live.

Troubleshooting

SYMPTOM	FIX
Subagent tools denied	union of the cast, plus hook
Silent empty research	`mcp_servers()` in Python
Fabricated sections	empty claims must escalate
Bill before a gate	check cost inside research and verify
Writer rewrote `paper.md`	deny anyone but writer, deny that path
SVG in the paper	renderer must exit 2, not substitute

Recap

A topic in. A paper plus a knowledge bundle out.

1. Trigger and runtime change. Exits do not.
2. `allowed_tools` gates the children.
3. MCP in Python, or a fresh clone silently cannot search.
4. Three budgets, and cost inside the phases.
5. `sourced` catches a citation a model will bless.

The grounding contract has to hold while the wiring is wrong.

sol4_fixer_agent_sdk

The working Module 4 loop. A failing branch in. A green one out, or an honest explanation.

`query()`, not `ClaudeSDKClient`. `permission_mode: dontAsk`. Merge is never a tool.

Saturday fills two stubs in `labs/lab4_fixer`. Demo `broken-pr` from here.

Read `HOW_TO_RUN.md`, `DESIGN_DOC.md`, and `E2E_PLAN.md`.

Layout

```
loop.py       summarize_failure + repair_until_green wrappers
fixer.py      the while loop
doers.py      none / reference / cli / SDK backend
gates.py      pass, retry, escalate
roles.py      dontAsk, one PreToolUse hook, deny tests/**
tests/        hook deny/allow, same-signature escalate
```

The Deep Agents twin is the graph, not the repair loop.

Setup. Stash first.

```
git -C ../../work/northwind-field-crm stash --include-untracked
cd solutions/sol4_fixer_agent_sdk
echo 'ANTHROPIC_API_KEY=sk-ant-...' >> ../../.env
task setup          # .venv + claude-agent-sdk. PEP 668.
task clone
task reset          # checkout broken-pr. Refuses a dirty clone.
```

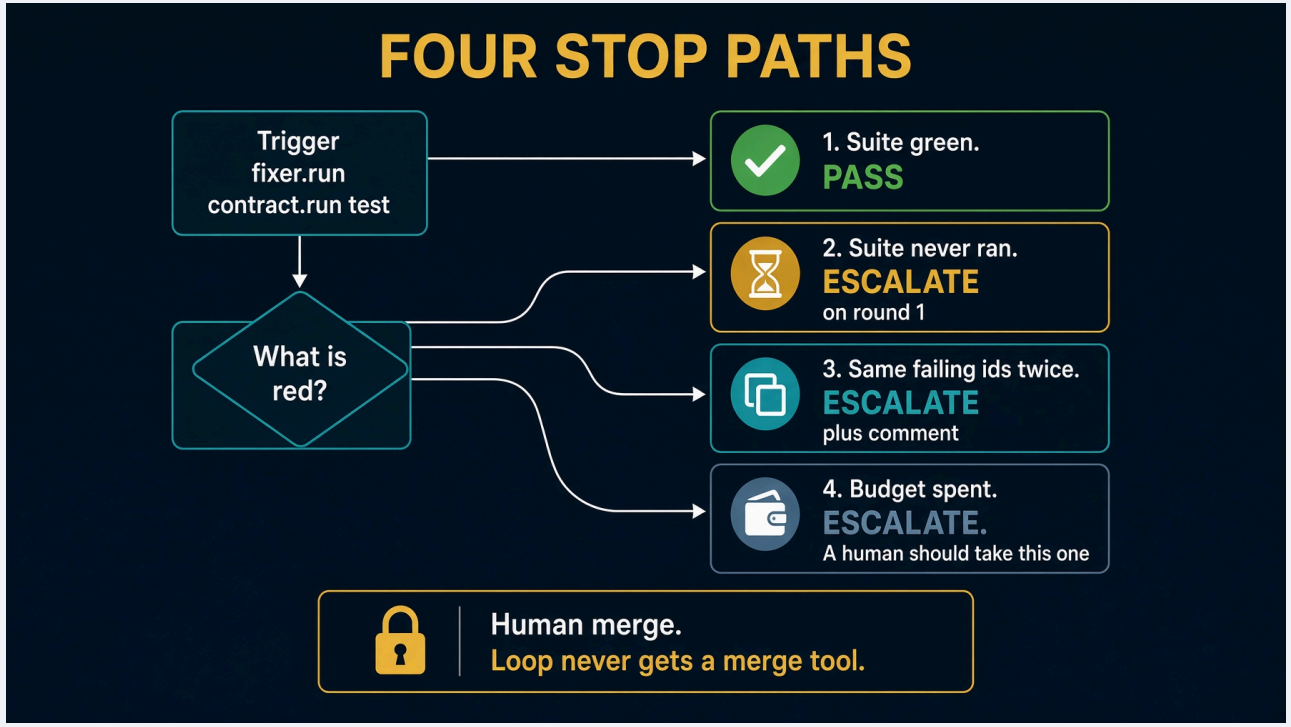
Say the stash out loud. That refusal is on brand.

Scripts with no model

```
task table          # judge writes must print no
task test
```

If judge prints `yes`, stop. Neither needs a key, the SDK, or a clone.

Architecture



Cast is three roles. No planner. See `docs/diagrams/architecture.svg`.

Five unattended lines

`acceptEdits` auto-accepts edits before the allow list. `dontAsk` denies anything not pre-approved.

Doers

SPEC	NEEDS	BEHAVIOR
`none`	clone	writes nothing. Loop still reports the truth
`reference`	clone	copies `known-good` inside `app/**`
`sdk`	`task setup` + key	`AgentSdkBackend` wrapping `query()`

```
task run -- --branch broken-pr --doer none
task run -- --branch broken-pr --doer reference
task run -- --branch broken-pr --doer sdk
```

`--research` defaults to `off`. `--doer sdk` refuses if you skipped `task setup`.

Expected `--doer none`

```
attempt 1: 1 failing -> retry
attempt 2: 1 failing -> escalate
gate: escalate
The fixer gave up.
A human should take this one.
```

`--doer reference` against `broken-pr`: `gate: pass`. Files copied from `known-good` inside `app/**` only.

What Python still owns

`summarize_failure` from junit. Four stop paths:

1. Suite green → pass
2. Suite never ran → escalate on round 1
3. Same failing ids twice → escalate, leave a comment
4. Budget spent → escalate, "A human should take this one"

Scope violations escalate even if the suite is green. Spend comes from `total_cost_usd`, so the money gate can fire.

E2E. Disposable clones

Do not use `work/northwind-field-crm` for a live probe. It may hold Lab 2 work.

`E2E_PLAN.md` clones into `/tmp`, prepares the target venv, then runs `--doer none` and `--doer reference` before `--doer sdk`.

`task setup` here installs the SDK. It does not install the CRM's test dependencies.

Troubleshooting

SYMPTOM	FIX
Dirty tree	stash, out loud
Green on `none`	same ids twice must escalate
Edited a test	full deny envelope on `tests/**`
`acceptEdits` in options	must be `dontAsk`
Thought DA repaired it	that twin is graph only

Recap

Same graph, nobody at the keyboard. Reference doer still bound by WriteScope. Human owns merge.

`dontAsk`. Deny `tests/**`. Leave a comment when you give up.

The loop is the product. The prompt is not.

Spillwave Solutions | spillwave.com

sol4_fixer_deep_agents

Configuration port. It does **not** run the fixer.

This is the role graph. The live harness is `sol4_fixer_agent_sdk`.

`task run` prints the cast and builds an agent. It never invokes a repair.

Read `HOW_TO_RUN.md` first. It overrides the DESIGN_DOC intro. Then read `E2E_PLAN.md`.

What it proves

```
role                writes  scope
orchestrator        no      nothing
code_implementer    yes     app/**, src/**   denied tests/**
judge               no      nothing
```

Missing on purpose: `fixer.py`, `doers.py`. The tests pin the graph.

Live repair is `sol4_fixer_agent_sdk`. `backend(contract)` exists. `main` never calls it.

Setup

```
cd solutions/sol4_fixer_deep_agents
cp config.json.example config.json
echo 'ANTHROPIC_API_KEY=sk-ant-...' >> ../../.env
task setup          # .venv + deepagents>=0.7
task clone
task reset          # checkout broken-pr. Refuses a dirty clone.
```

`task reset` still matters. A dirty clone is Lab 2 work you must not delete.

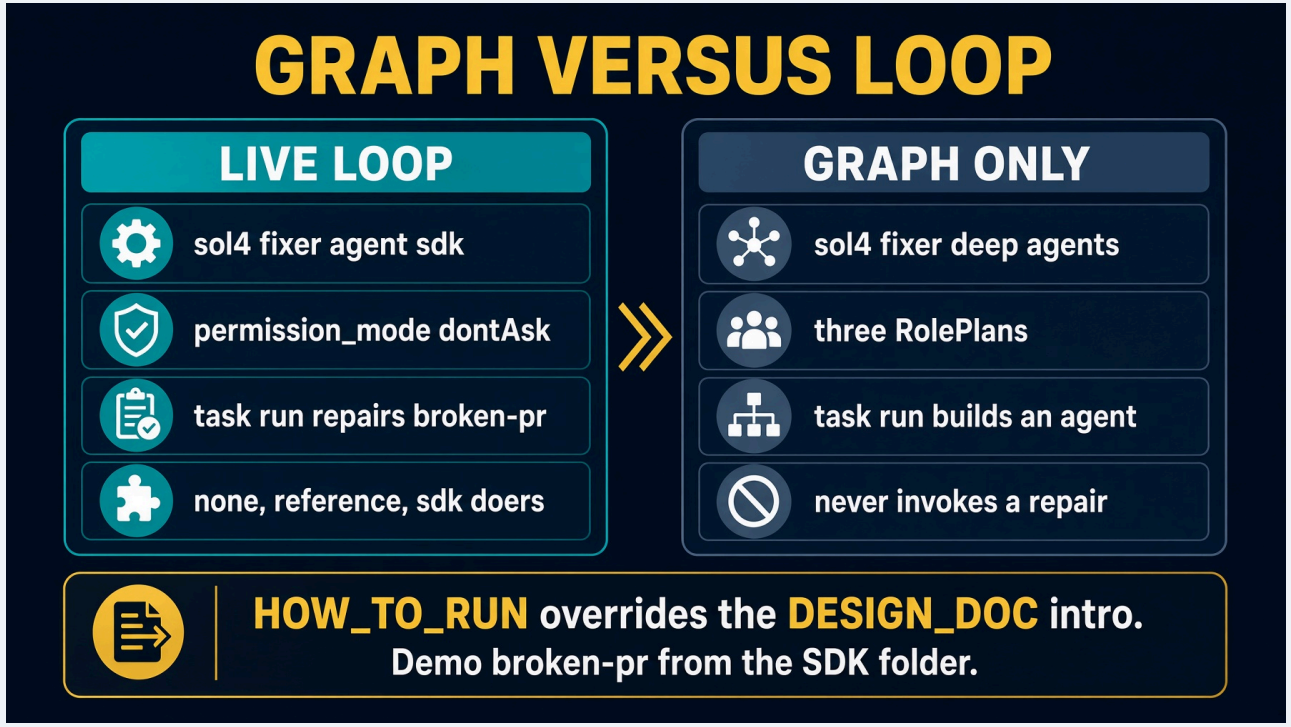
Scripts with no model

```
task table          # judge writes must print no
task test
```

If judge prints `yes`, stop. Neither needs Deep Agents, a key, or a clone.

Missing repo plus `--table-only`: prints declared scopes, exit 0. Without `--table-only`, `ContractError` raises.

Architecture



No while loop. No junit. HOW_TO_RUN overrides the DESIGN_DOC intro.

Three fences. Same as the other DA ports.

1. A tool list per subagent. Judge gets `read_file` only.
2. A path check inside the write tool. Refusal is a sentence.
3. A fence around the harness. Virtual filesystem. General-purpose subagent **off**.

```
@tool(f"write_{role.name}")
def write(path: str, content: str) -> str:
    try:
        scope.check(path)
    except ScopeViolation:
        return f"REFUSED. {role.name} may write {allowed}. {path} is outside that scope."
```

Skills and memory are mounts

`skills/` and `memory/` are not a fourth fence.

A skill loads its instructions when the role is invoked, rather than sitting in every prompt from the start.

`/memory/` routes at `memory/`, not this folder. Routing it at the solution root would hand the agent this folder's own source.

Why three roles, not five

The work is already defined by what is red. No planner. No test implementer.

The judge reads junit. The coder may write `app/**` and may not write `tests/**`.

`DEFAULT_LOOP` and `LOOP` are both `"fixer"`. A caller still names its loop at every site. A test in the folder pins both.

Commands

```
task table
task test
task run --
```

`task run` needs `task setup` and the clone. Extra flags after `--` go to `loop.py`.

This folder will not edit a failing test into passing. It will not paste `SKILL.md` into a subagent prompt.

Troubleshooting

SYMPTOM	FIX
Expected a green branch	config port. Use the Agent SDK folder
Judge writes `yes`	tools = `[read_file]`
Raises on missing repo	`--table-only` is the self-check
Dirty clone on reset	stash Lab 2 work first
DESIGN_DOC says it repairs	HOW_TO_RUN is the operator contract

Recap

Same table, Deep Agents enforcement knob. This is graph without a loop.

Use `sol4_fixer_agent_sdk` when you want a green branch or an honest comment.

`task setup`, `task table`, `task test`, `task run`. Read `HOW_TO_RUN.md`.

Extra credit 1. The webhook receiver

Not Saturday. Do not skip Module 2 for this.

New trigger. Same exits. Same harness. The graph does not change.

One FastAPI server GitHub can POST to. Extra credit 2 and 5 both need it.

Filled answer: `solutions/extra_credit/s_ext_1_webhook/`

What you will build

A receiver that **starts** one poll. It does not grade the ticket.

ROUTE	JOB
`GET /health`	backend name and sol1 path
`POST /github-webhook`	verify HMAC, reply 202, spawn `task run`

It shells out to `solutions/sol1_enhancer`. It does **not** import it.

Stub: `labs/extra-credit/ext_1_webhook/webhook_server.py`

Why this exists

`task poll-forever` is a seminar lie. Close the laptop and polling stops.

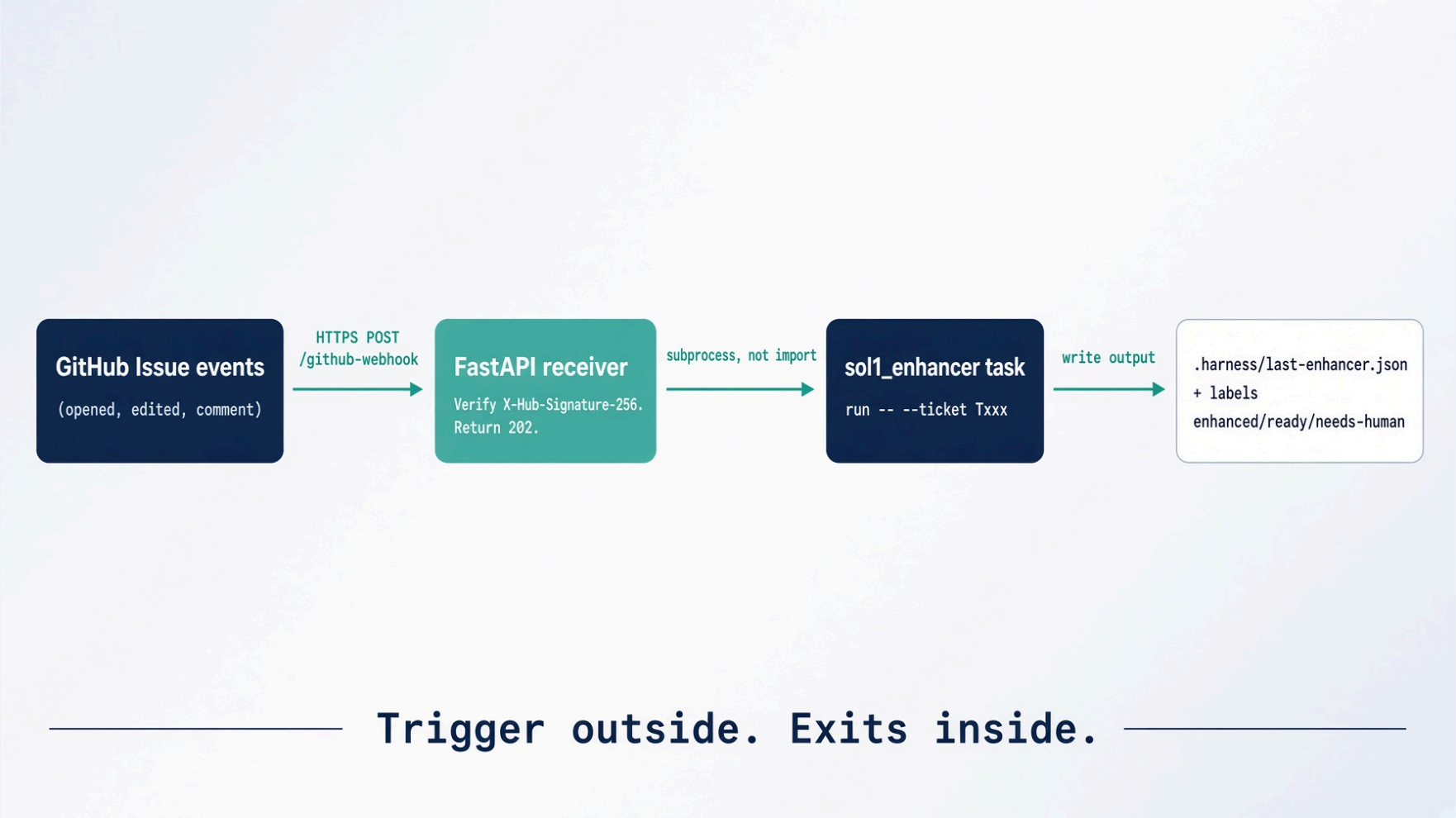
Production is an event. GitHub already knows when a ticket changed.

The trigger moves out of the loop. The exits stay in it. A webhook starts the run. It never decides when to stop.

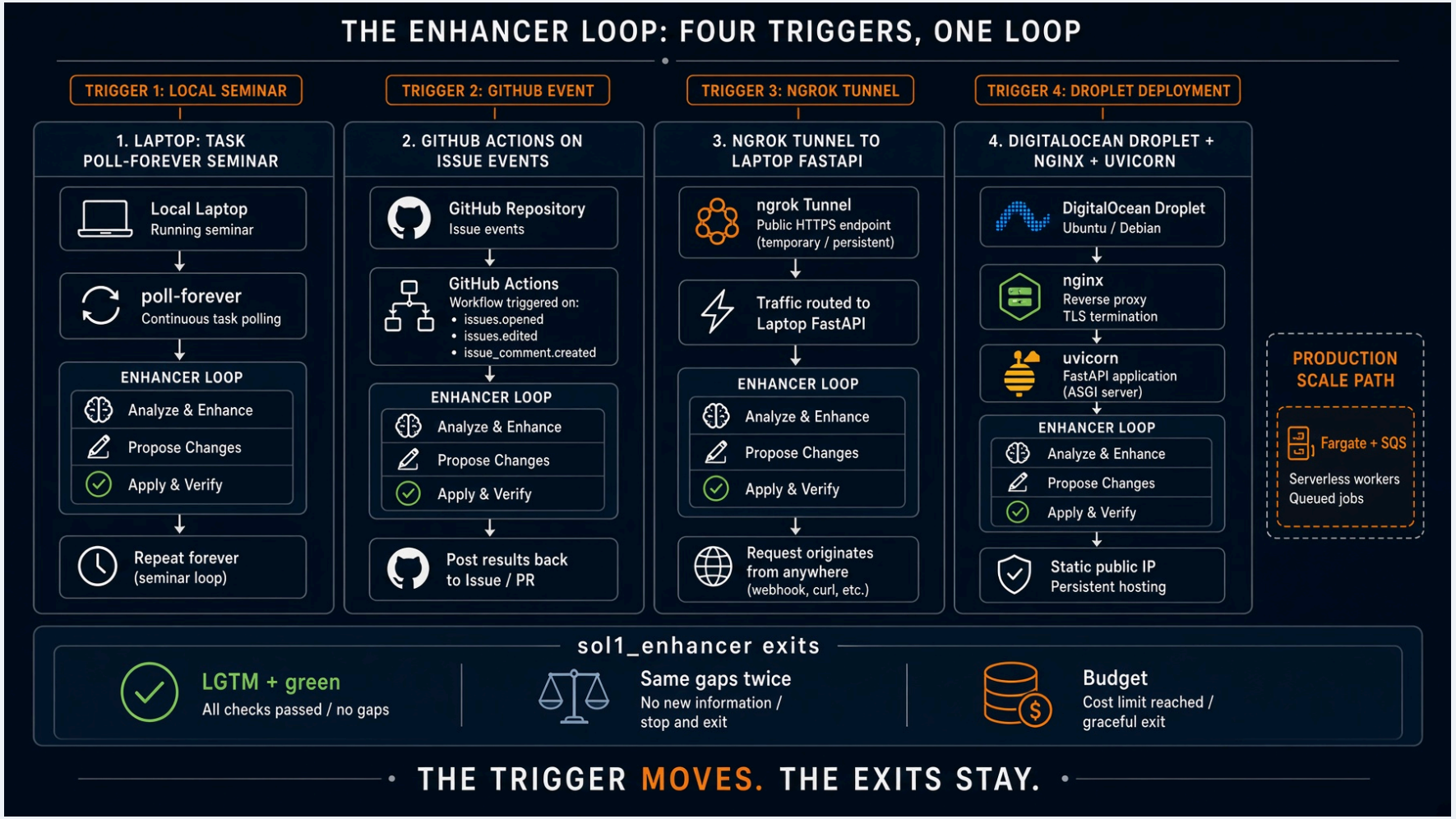
Learning objectives

- Serve `POST /github-webhook` and `GET /health`
- Verify `X-Hub-Signature-256` on the **raw** body
- Reply 202 before Claude starts
- Lock one issue at a time
- Map `[T001]` in the title to `--ticket T001`
- Skip comments that contain `<!-- enhancer-loop -->`
- Journal every delivery to `work/last-webhook.json`

Starting architecture



The rule on one slide



Same `sol1_enhancer` exits under every trigger:

1. `LGTM` plus a green rubric
2. Same missing fields twice

Prerequisites

```
cd /path/to/reliable-agentic-lab
python -m uvicorn solutions.extra_credit.s_ext_1_webhook.webhook:app \
  --host 127.0.0.1 --port 8000
curl -s localhost:8000/health
```

Need: FastAPI, uvicorn, `task` on PATH, a filled `solutions/sol1_enhancer`.

The lab stub re-exports that module so extra credit 2 and 5 can keep pointing at `labs/extra-credit/ext_1_webhook/webhook_server.py`.

Step 1. Two routes. Secret first.

Missing secret is 503, not 401. An empty secret is not "unsigned is fine".

```
def secret() -> str:
    return (os.environ.get("GITHUB_WEBHOOK_SECRET") or "").strip()

@app.get("/health")
def health() -> dict:
    return {"ok": True, "backend": backend_name(), "sol1": str(call_sol1.sol1_dir())}
```

Step 2. HMAC on the raw body

```
def verify_signature(body: bytes, header: str | None) -> None:
    value = secret()
    if not value:
        raise HTTPException(status_code=503, detail="GITHUB_WEBHOOK_SECRET is not set")
    if not header:
        raise HTTPException(status_code=401, detail="missing X-Hub-Signature-256")
    digest = hmac.new(value.encode("utf-8"), body, hashlib.sha256).hexdigest()
    expected = "sha256=" + digest
    if not hmac.compare_digest(expected, header.strip()):
        raise HTTPException(status_code=401, detail="bad signature")
```

Never verify against re-serialized JSON. Byte order changes. HMAC then fails for real GitHub too.

Step 3. 202, then work in the background

GitHub waits about 10 seconds. Claude takes minutes.

```
body = await request.body()
verify_signature(body, x_hub_signature_256)
payload = json.loads(body.decode("utf-8") or "{}")
write_journal({..., "status": "accepted"})
background.add_task(handle_delivery, event, payload)
return JSONResponse(preview, status_code=202)
```

A crash in the worker is journaled. It must not become a 500 back to GitHub. GitHub would retry and you would pay twice.

Step 4. Route. Only groom is wired

```
if event == "issues" and action == "opened":
    return "groom"
if event == "issue_comment" and action == "created":
    if "<!-- enhancer-loop -->" in body:
        return None
    return "groom"
if event == "issues" and action == "labeled" and name == "ready":
    return "fulfill"   # not wired. journal says so.
if event == "check_suite" and conclusion == "failure":
    return "fix"       # not wired. Saturday Lab 4.
```

Fulfill and fix stay Saturday labs. Extra credit 1 only starts the enhancer.

Step 5. One lock. Attempt budget. In-progress.

```
if not acquire_issue(number):
    return {"status": "busy"}
if attempts >= max_attempts():
    client.comment(number, f"Giving up after {attempts} attempts ...")
    return {"status": "gave-up"}
client.add_label(number, "agent-in-progress")
try:
    result = call_sol1.run_sol1(ticket_id)
finally:
    client.remove_label(number, "agent-in-progress")
```

`AGENT_MAX_ATTEMPTS` default 3. Second belt. The loop already has a round budget.

Step 6. Ticket id. Do not invent one.

```
TICKET_IN_TITLE = re.compile(r"\[(T\d+)\]", re.IGNORECASE)
TICKET_IN_BODY  = re.compile(r"(?:^|\n)\s*id:\s*(T\d+)\b", re.IGNORECASE)
```

Title `[T001] ...` is enough. Frontmatter `id: T001` also works. No id: comment and stop.

Step 7. Shell out. Never import.

```
BACKEND_FOLDERS = {
    "claude": "sol1_enhancer",
    "grok": "sol1_enhancer_grok_build",
    "opencode": "sol1_enhancer_opencode",
    "codex": "sol1_enhancer_codex",
    "agent-sdk": "sol1_enhancer_agent_sdk",
    "deep-agents": "sol1_enhancer_deep_agents",
}
cmd = ["task", "run", "--", "--ticket", ticket_id]
subprocess.run(cmd, cwd=str(sol1_dir()), timeout=AGENT_TIMEOUT)
```

`AGENT_BACKEND` picks the folder. Default `claude`. Timeout default 900s.

Commands and expected result

```
export GITHUB_WEBHOOK_SECRET=pick-a-long-random-string
python -m uvicorn solutions.extra_credit.s_ext_1_webhook.webhook:app --port 8000
curl -s localhost:8000/health
python -m pytest solutions/extra_credit/s_ext_1_webhook/tests -q
```

Health: `{"ok": true, "backend": "claude", "sol1": ".../sol1_enhancer"}`.

Tests use `fake_github.py`. No token. The sol1 handoff is monkeypatched.

Troubleshooting

SYMPTOM	CAUSE	FIX
503	secret empty	export `GITHUB_WEBHOOK_SECRET`
401	secret mismatch, or verified parsed JSON	same string as GitHub. HMAC raw bytes.
202, no Claude	`task` not on PATH	install Task, check journal `stderr`
Gave-up comment	`AGENT_MAX_ATTEMPTS` hit	lower it while testing, or reset labels
Loop replies forever	marker filter missing	skip `<!-- enhancer-loop -->`

Validation checklist

- [ ] `/health` names the sol1 folder
- [ ] Unsigned POST is 401
- [ ] Missing secret is 503
- [ ] Signed `issues`/`opened` is 202
- [ ] Journal has `status: accepted` then a later `ok` / `sol1-failed`
- [ ] `call_sol1` never `import`s the enhancer package
- [ ] Fulfill and fix routes journal `not-wired`

Recap

What we built. FastAPI that verifies, locks, journals, and starts one poll.

Takeaways

1. Trigger outside. Exits inside.
2. HMAC on the raw body. `compare_digest`.
3. 202 before the model starts.
4. Marker, not author, when reading comments.
5. Next: tunnel it with ngrok, or put it on a Droplet.

Extra credit 2. ngrok adapter

Not Saturday. Run the Lab 1 enhancer on your laptop. GitHub still sends real events.

New trigger. Same exits. Same harness. The graph does not change.

Answer: `solutions/extra_credit/s_ext_2_ngrok/`

No FastAPI. Stdlib `ThreadingHTTPServer`. Port 8765.

What you will build

1. Copy the Lab 1 plugin into this folder (`task copy-plugin`)
2. Configure `config.json` and clone your CRM fork
3. Run `bin/webhook_trigger.py`
4. Give GitHub a public URL with ngrok
5. Open an issue titled `[T900] ...` and watch one `task run` start

The adapter is not the loop. It verifies, replies 202, and spawns `task run -- --ticket Txxx`.

Why ngrok

GitHub needs a public HTTPS URL. Your laptop only has localhost.

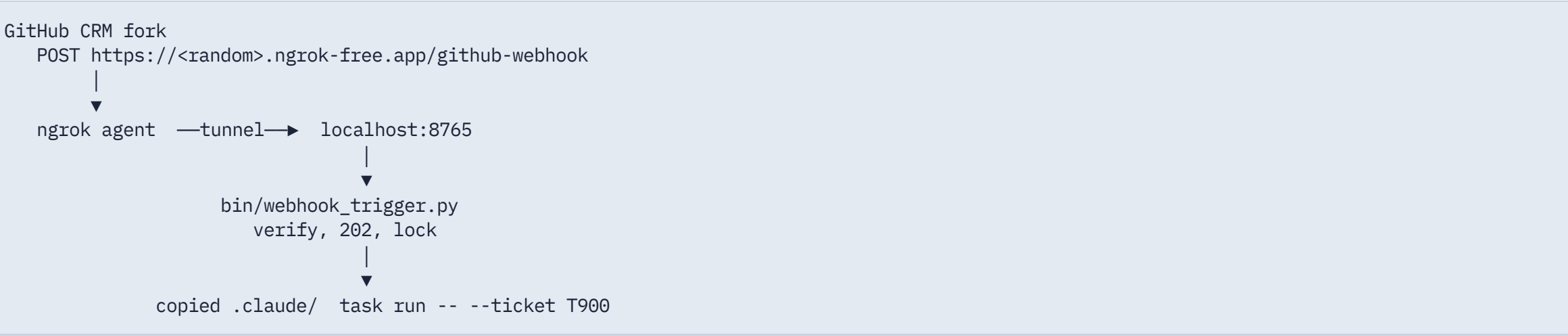
ngrok opens an outbound tunnel. You do not open a firewall port.

Free tier (August 2026): 3 endpoints, 1 GB, 20k HTTP requests. URLs change on restart. Paid reserved domains stay put.

Learning objectives

- Copy the plugin without importing `sol1_enhancer` at runtime
- Listen on 8765 so the CRM on 8000 is free
- Verify HMAC in Python even if ngrok also checks at the edge
- Dedup on `X-GitHub-Delivery`
- Prove a live GitHub delivery with 202 and `work/last-webhook.json`

Starting architecture



Step 1. Copy the plugin here

```
cd solutions/extra_credit/s_ext_2_ngrok
task copy-plugin
```

Copies `.claude/`, `config.json.example`, `bin/setup_test_tickets.sh`.

Does **not** copy `SPEC.md`, `HOW_TO_RUN.md`, or Lab 1's Taskfile. This folder owns those. Copied `.claude/` is gitignored.

Step 2. Configure and prove one poll by hand

```
cp config.json.example config.json   # set fork_owner
task clone
task create-test-tickets             # optional T900 T901 T902
task run -- --ticket T001
```

That is the exact command the adapter will spawn. If this fails, ngrok will not save you.

Step 3. Three terminals

Terminal A:

```
export GITHUB_WEBHOOK_SECRET='pick-a-long-random-string'
task listen                          # python3 bin/webhook_trigger.py
```

Terminal B:

```
ngrok http 8765
```

Copy the HTTPS URL. Webhook path is `/github-webhook`.

Terminal C:

```
curl -s http://127.0.0.1:8765/health
```

Want `"ok": true` and `"plugin_ready": true`.

Register the webhook on the CRM fork

On **northwind-field-crm**, not this lab repo.

- Payload URL: `https://YOUR-SUBDOMAIN.ngrok-free.app/github-webhook`
- Content type: `application/json`
- Secret: same `GITHUB_WEBHOOK_SECRET`
- Events: Issues, Issue comments. Nothing else.

GitHub sends `ping`. Adapter returns `{"status":"pong"}`. Deliveries tab should show 200.

What the adapter does. Eleven rules.

1. `GET /health` reports whether the copied plugin is present
2. Read the raw body first
3. HMAC SHA-256 vs `X-Hub-Signature-256`. Missing secret 503. Bad sig 401
4. `ping` is `pong`
5. Title must contain `[Txxx]`
6. `issues` opened, edited, reopened starts one poll
7. `issue_comment` created starts one poll unless the marker is in the body
8. Reply 202 before Claude starts
9. One lock file per ticket under `work/locks/`
10. One file per `X-GitHub-Delivery` under `work/deliveries/`
11. Write `work/last-webhook.json` on every accepted run

Signature and marker. Real code.

```
def verify_signature(payload: bytes, signature: str, secret: str) -> bool:
    if not secret or not signature:
        return False
    if not signature.startswith("sha256="):
        return False
    expected = "sha256=" + hmac.new(secret.encode(), payload, hashlib.sha256).hexdigest()
    return hmac.compare_digest(expected, signature)
```

```
MARKER = "<!-- enhancer-loop -->"
if MARKER in comment:
    return None
```

Dedup and lock

```
def already_seen(delivery_id: str) -> bool:
    path = work_dir() / "deliveries" / f"{delivery_id}.json"
    if path.exists():
        return True
    path.write_text("{}", encoding="utf-8")
    return False

def acquire_lock(ticket: str) -> bool:
    fd = os.open(lock_path(ticket), os.O_CREAT | os.O_EXCL | os.O_WRONLY)
```

GitHub retries. At-least-once is the contract. The delivery id file is the idempotency key.

Local HMAC smoke test

```
BODY='{"action":"opened","issue":{"number":1,"title":"[T900] grey button"}}'
SIG="sha256=$(python3 -c '...hmac...')"
curl -s -D- -X POST http://127.0.0.1:8765/github-webhook \
  -H "Content-Type: application/json" \
  -H "X-GitHub-Event: issues" \
  -H "X-GitHub-Delivery: local-test-1" \
  -H "X-Hub-Signature-256: $SIG" \
  --data "$BODY"
```

Unsigned posts must return 401.

End to end

1. Open `[T900] login button is grey` on the CRM fork
2. GitHub posts `issues`/`opened`. Adapter replies 202
3. `task run -- --ticket T900` grooms, posts a marked comment
4. You reply `LGTM` after the rubric is green
5. Next poll sets `ready` and `loop: implementer`

Watch: GitHub deliveries, ngrok inspector `http://127.0.0.1:4040`, `work/last-webhook.json`, `work/enhancer-T900.log`.

Troubleshooting

SYMPTOM	CAUSE	FIX
Delivery is HTML	free interstitial	paid domain, or confirm User-Agent is GitHub-Hookshot
401	secret mismatch	same string in GitHub UI and env
503 plugin not copied	`.claude/` missing	`task copy-plugin`
202, no Claude	`task` not on PATH	read `work/enhancer-Txxx.log`
ngrok URL 404 after restart	free URL rotated	paste the new URL into the webhook
Loop replies forever	missing marker filter	adapter drops the marker

Recap

This is still a laptop demo. Close the lid and the tunnel dies.

Production is GitHub Actions, a Droplet, or Fargate. Same one-shot skill. Same state file in `.harness/`.

Extra credit 5. DigitalOcean Droplet

Not Saturday. A cheap permanent public endpoint.

New trigger. Same exits. Same harness. The graph does not change.

Cheapest Basic plan. Ubuntu 24.04 LTS. One vCPU. 1 GB RAM. About six dollars a month.

No new Python. You put extra credit 1 on a box.

What you will build

The receiver is `solutions/extra_credit/s_ext_1_webhook`. nginx is the only public door. systemd runs uvicorn on loopback.

PIECE	PATH
`.env.example`	`labs/extra-credit/ext_5_digitalocean/.env.example`
systemd unit	`deploy/agent-webhook.service`
nginx	`deploy/nginx.conf`
sol1 config	`deploy/write-sol1-config.sh`
handoff	`deploy/call-sol1-enhancer.sh T001`
smoke	`deploy/smoke.sh`

Why bind to localhost

The agent holds a token. A default bind of `0.0.0.0` puts it on the public internet.

```
Internet ──TLS──▶ nginx :443 ──proxy──▶ 127.0.0.1:8000 uvicorn
```

`ProtectSystem=strict`. `NoNewPrivileges=yes`. User `agent`.

Learning objectives

- Bootstrap Ubuntu 24.04 with one script
- Fill `/opt/agents/.env` from the example
- Terminate TLS with certbot
- Point the **CRM fork** webhook at `https://your-domain/github-webhook`
- Smoke-test HMAC locally before GitHub
- Swap `AGENT_BACKEND` without rewriting the unit

Starting architecture

```
GitHub CRM fork
   POST https://agents.example.com/github-webhook
            |
            ▼
        nginx (TLS)
            |  proxy_read_timeout 120s
            ▼
     127.0.0.1:8000  s_ext_1_webhook
            |  subprocess
            ▼
     solutions/sol1_enhancer   (or grok / opencode / ...)
```

One command on the box

```
ssh root@YOUR_DROPLET_IP
git clone https://github.com/YOUR-FORK/reliable-agentic-lab.git /opt/agents
bash /opt/agents/labs/extra-credit/ext_5_digitalocean/deploy/bootstrap.sh
```

Then edit `/opt/agents/.env`. Point DNS. `certbot --nginx -d your-domain`.

What bootstrap installs

- python3, venv, nginx, certbot, git, jq, `task`
- system user `agent`
- clone or pull `/opt/agents`
- venv at `/opt/agent-env` with `requirements.txt`
- `.env` from the example if missing (`chmod 0600`)
- optional clone of the CRM fork into `work/northwind-field-crm`
- `write-sol1-config.sh` then `install.sh`

`.env` that matters

```
GITHUB_TOKEN=
GITHUB_REPO=your-github-username/northwind-field-crm
GITHUB_WEBHOOK_SECRET=
AGENT_BACKEND=claude
AGENT_MAX_ATTEMPTS=3
AGENT_TIMEOUT=900
WEBHOOK_HOST=127.0.0.1
WEBHOOK_PORT=8000
ANTHROPIC_API_KEY=
WEBHOOK_DOMAIN=agents.example.com
```

`GITHUB_REPO` is the CRM fork, **not** the lab repo.

`AGENT_BACKEND` picks the folder

```
claude      -> solutions/sol1_enhancer
grok        -> solutions/sol1_enhancer_grok_build
opencode    -> solutions/sol1_enhancer_opencode
codex       -> solutions/sol1_enhancer_codex
agent-sdk   -> solutions/sol1_enhancer_agent_sdk
deep-agents -> solutions/sol1_enhancer_deep_agents
```

Grok is a good fit on a Droplet (you can `task trust` once). It is a poor fit on hosted Actions.

systemd unit. Loopback only.

```
ExecStart=/opt/agent-env/bin/python -m uvicorn \
  solutions.extra_credit.s_ext_1_webhook.webhook:app \
  --host 127.0.0.1 --port 8000
User=agent
EnvironmentFile=/opt/agents/.env
ReadWritePaths=/opt/agents
```

Working directory `/opt/agents`. `PYTHONPATH=/opt/agents`.

nginx. Two locations. Everything else 404.

```
location /health         { proxy_pass http://127.0.0.1:8000/health; }
location /github-webhook {
    proxy_pass http://127.0.0.1:8000/github-webhook;
    proxy_read_timeout 120s;
}
location / { return 404; }
```

`client_max_body_size 1m`. Replace `YOUR_DOMAIN` before certbot.

GitHub webhook

On the CRM fork (`GITHUB_REPO`):

- Payload URL: `https://your-domain/github-webhook`
- Content type: `application/json`
- Secret: same as `GITHUB_WEBHOOK_SECRET`
- Events: Issues, Issue comments, Check suites

Issue titles must include `[T001]`.

Smoke without GitHub

```
source /opt/agents/.env
bash labs/extra-credit/ext_5_digitalocean/deploy/smoke.sh
sudo -u agent bash -lc 'source /opt/agents/.env && \
  /opt/agents/labs/extra-credit/ext_5_digitalocean/deploy/call-sol1-enhancer.sh T001 --print'
```

`smoke.sh` GETs `/health`, then POSTs a signed `issues.opened` body at loopback.

Expected live result

A 202 in the GitHub webhook log. A record in:

```
/opt/agents/solutions/extra_credit/s_ext_1_webhook/work/last-webhook.json
```

A `task run` in `solutions/sol1_enhancer` for that ticket.

```
journalctl -u agent-webhook -f
```

Troubleshooting

SYMPTOM	CAUSE	FIX
Droplet 502	uvicorn down, or proxy_pass wrong	`systemctl status agent-webhook`
401	secret mismatch	same string in GitHub and `.env`
503	secret empty	fill `.env`, restart unit
Clone failed	`CRM_OWNER` still the placeholder	edit `.env`, run `write-sol1-config.sh`
Grok skill not found	untrusted git root	`task trust` as the `agent` user

Recap

Permanent HTTPS. Same receiver as extra credit 1. Same exits as Lab 1.

The lid can close. The box keeps listening.

Next scale step: Fargate plus a queue. Same HMAC. Same 202. Worker is a second service.

Deploy on GitHub Actions

Ticket change events. One poll. Then exit.

New trigger. Same exits. Same harness. The graph does not change.

Copy-me workflow: `labs/lab1_enhancer/workflows/enhance-on-issue.yml`

Notes: `labs/lab1_enhancer/GITHUB-ACTIONS.md`

Not Saturday. Copy onto **your CRM fork**. Never the instructor repo.

What you will deploy

A workflow that GitHub already knows how to start.

```
on:
  issues:
    types: [opened, edited, labeled]
  issue_comment:
    types: [created]
  workflow_dispatch:
```

That is "the ticket changed": a new issue, an edited body, a label, or a human comment. `LGTM` arrives as `issue_comment`.

Why Actions instead of polling

`task poll-forever` dies when the laptop sleeps.

GitHub already emitted the event. Hosting a receiver is optional.

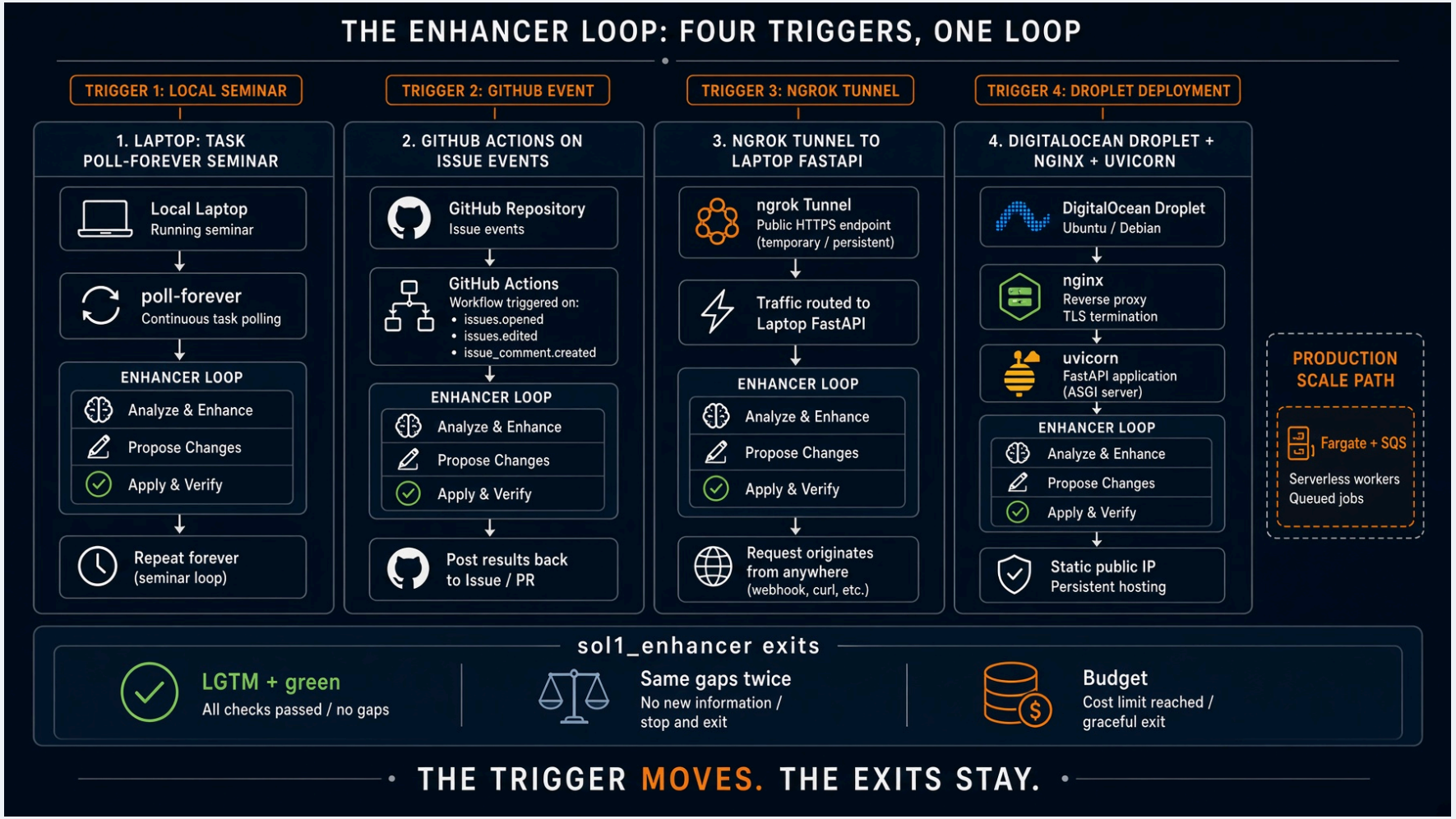
The trigger starts one poll. The enhancer exits. The workflow does not grade the ticket, merge, or deploy.

Learning objectives

- Copy the YAML onto the CRM fork
- Parse `[T900]` from the issue title
- Skip `<!-- enhancer-loop -->`
- Serialize with `concurrency.group` per issue
- Swap `ENHANCER_BACKEND` without rewriting jobs
- Know which backends do **not** belong on hosted runners

Starting architecture

Four triggers. Same exits.



Actions is column two. The loop is still `sol1_*`.

Copy onto YOUR fork

```
mkdir -p .github/workflows
cp path/to/reliable-agentic-lab/labs/lab1_enhancer/workflows/enhance-on-issue.yml \
  .github/workflows/enhance-on-issue.yml
```

Do not enable this on the shared instructor repo. It comments on issues and can spend model tokens.

Permissions. Least you need.

```
permissions:
  issues: write
  contents: read
  actions: read
```

Grant `contents: write` only if a later job will push. The enhancer edits issue bodies and labels, not app code.

A workflow that merges has left the lab.

Concurrency per issue

```
concurrency:
  group: enhancer-${{ github.event.issue.number || github.event.inputs.ticket }}
  cancel-in-progress: false
```

Two events on the same ticket must not chew it twice. Do not cancel in progress. A mid-poll kill leaves labels and a half-written candidate.

Skip the loop's own comments

```
if: >
  github.event_name == 'workflow_dispatch' ||
  github.event_name == 'issues' ||
  (github.event_name == 'issue_comment' &&
   !contains(github.event.comment.body, '<!-- enhancer-loop -->'))
```

Filter on the marker, never the author. The job runs as `github-actions[bot]`. A human `LGTM` must still count.

Ticket id from the title

```
id="$(printf '%s
' "$TITLE" | grep -oE '\[T[0-9]+\]' | head -1 | tr -d '[]')"
if [ -z "$id" ]; then
  id="T${NUMBER}"
fi
```

Seed titles look like `[T900] Search crashes on an empty query`. If the title has no id, `T{issue_number}` matches HOW_TO_RUN for issues opened in the UI.

Backend matrix

ENHANCER_BACKEND	FOLDER	HOW THE JOB STARTS IT
`claude` (default)	`sol1_enhancer`	`task run -- --ticket "$TICKET"`
`codex`	`sol1_enhancer_codex`	same
`opencode`	`sol1_enhancer_opencode`	same
`agent-sdk`	`sol1_enhancer_agent_sdk`	`python3 loop.py --once ...`
`deep-agents`	`sol1_enhancer_deep_agents`	`python3 loop.py --once ...`

Set repository variable `ENHANCER_BACKEND`. Secret `ANTHROPIC_API_KEY` for SDK and Deep Agents.

What does not belong on hosted runners

Grok Build. Trust prompt. Local plugin shims. `task trust` is a laptop step.

Keep Grok on a Droplet (`ext_5`) or a laptop (`ext_2`).

Codex needs `codex` on the runner. If the image does not have it, use Agent SDK or Claude Code.

In-progress and the second belt

```
gh issue edit "$NUMBER" --add-label agent-in-progress
# ... one poll ...
gh issue edit "$NUMBER" --remove-label agent-in-progress   # if: always()
```

Stop at `AGENT_MAX_ATTEMPTS` (default 3) by counting `agent-attempts-*` labels. The loop already has a round budget. This is a second belt, at the trigger.

Two checkouts

```
- uses: actions/checkout@v4          # the CRM fork, $GITHUB_WORKSPACE
- uses: actions/checkout@v4
  with:
    repository: ${{ vars.ENHANCER_REPO || github.repository }}
    path: .enhancer
```

`--repo "$GITHUB_WORKSPACE"` points the enhancer at the CRM files. The enhancer code lives in `.enhancer`.

Run one poll

```
case "$ENHANCER_BACKEND" in
  claude)    cd solutions/sol1_enhancer && task run -- --ticket "$TICKET" --repo "$TARGET" ;;
  agent-sdk) cd solutions/sol1_enhancer_agent_sdk && python3 loop.py --once --repo "$TARGET" --ticket "$TICKET" ;;
  deep-agents) cd solutions/sol1_enhancer_deep_agents && python3 loop.py --once --repo "$TARGET" --ticket "$TICKET" ;;
esac
```

Upload `.harness/last-enhancer-*.json` as an artifact. That is how you read the last score at 2 a.m.

What does not change

Three exits, no fourth: `LGTM` plus a green rubric, same gaps twice, budget spent.

Labels `enhanced`, `ready`, `needs-human`. Judge still has no write tool. `check_fields.py` still computes ready.

Verify

1. Push the workflow to **your** fork
2. Open `[T900] Search crashes on an empty query`
3. Actions tab shows `enhance`. Issue gets `enhanced` or a marked comment
4. Comment `LGTM` as yourself. Next event sets `ready` only if the rubric is already green
5. A second event with no new human comment must be a no-op. If it posts again, the marker filter is missing

Troubleshooting

SYMPTOM	CAUSE	FIX
Never fires	workflow on the instructor repo	copy YAML to your fork
Loops	marker filter missing	skip `<!-- enhancer-loop -->`
404 on issues	token cannot write issues	`issues: write`, or a PAT for a private CRM
Grok skill not found	hosted runner	use `claude` / `agent-sdk` / `deep-agents`
No artifact	harness path wrong	upload `**/.harness/last-enhancer-*.json`

When Actions is the wrong tool

Laptop demo: extra credit 2, ngrok.

Permanent cheap box: extra credit 5, Droplet.

Long-running model calls, private VPC, or you already live on AWS: Fargate. Next deck.

If you stall, return to `task run --` from `labs/lab1_enhancer`. That is the Saturday path.

Recap

Takeaways

1. Copy onto the CRM fork.
2. Trigger starts. Loop stops.
3. Marker, not author.
4. Concurrency per issue. Never cancel mid-poll.
5. Grok stays off hosted runners.

Closing line. The workflow is a doorbell. It is not the loop.

Deploy on AWS Fargate

Production pattern. **Not a shipped lab.** There is no `ext_fargate` folder.

Reuses extra credit 1: verify HMAC, reply 202, then run `sol1_enhancer`.

The new knob is a queue. GitHub's 10-second budget cannot wait for Claude.

What you will build (architecture, not a drop-in)

PIECE	JOB
ALB	public HTTPS, GitHub only
Fargate service `webhook-receiver`	verify, dedup, enqueue, 202
SQS	at-least-once buffer
Fargate service `enhancer-worker`	`task run -- --ticket Txxx`
Secrets Manager	webhook secret, `GITHUB_TOKEN`, model keys
DynamoDB	delivery-id idempotency
CloudWatch	journal you can read at 2 a.m.

Same exits as Lab 1. The trigger moved again.

New trigger. Same exits. Same harness. Fargate splits receive from work. That is still not a new loop.

Why Fargate, not Lambda

A poll can run 15 minutes (`AGENT_TIMEOUT=900`). Lambda's practical ceiling is a poor fit once the model is in the loop.

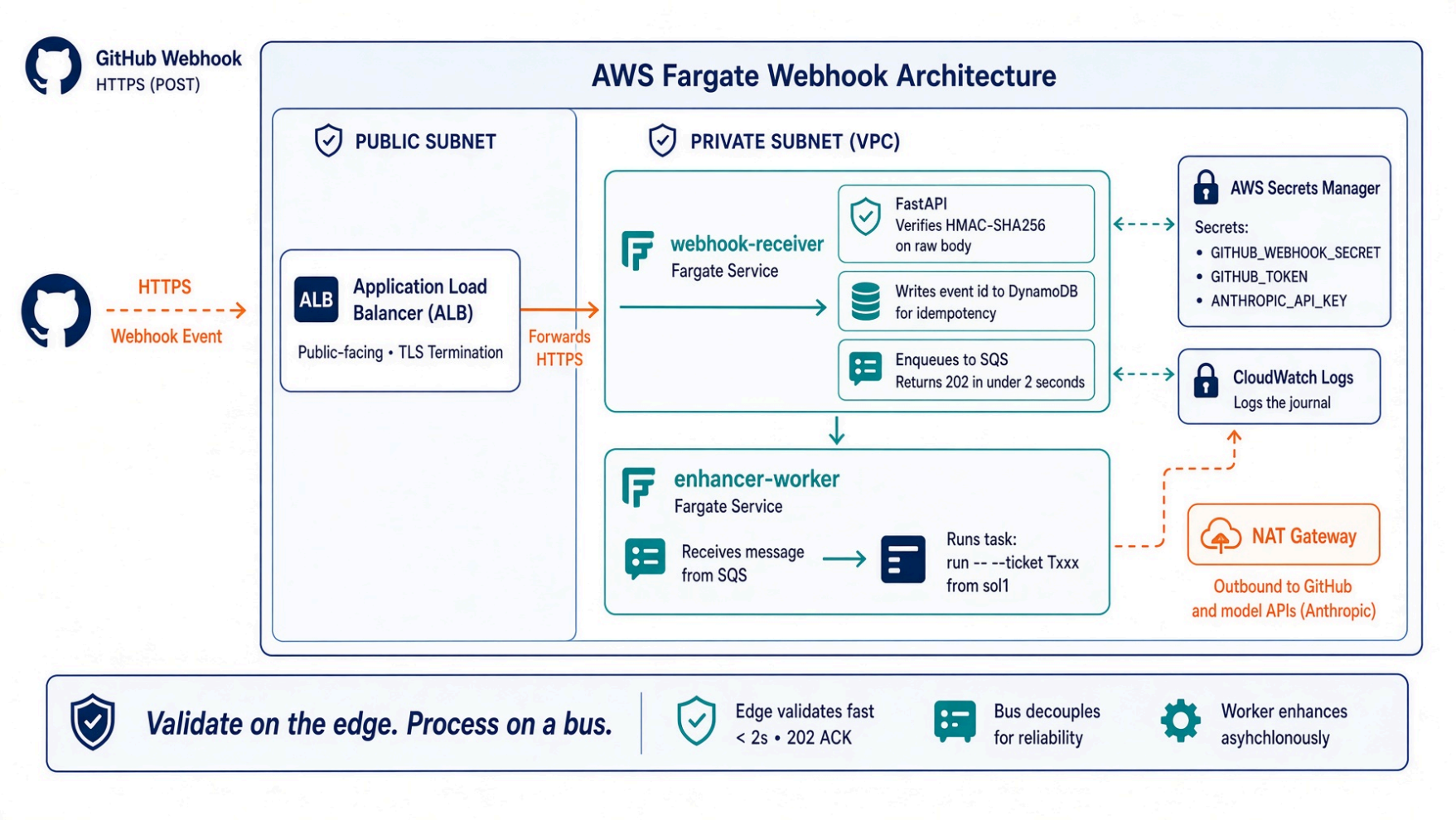
Fargate task CPU and memory are sized for the **worker**, not the receiver. The receiver should be tiny. The worker should be the size of Claude plus git.

GitHub webhooks: return 2xx fast. Process on a bus. (2026 webhook practice.)

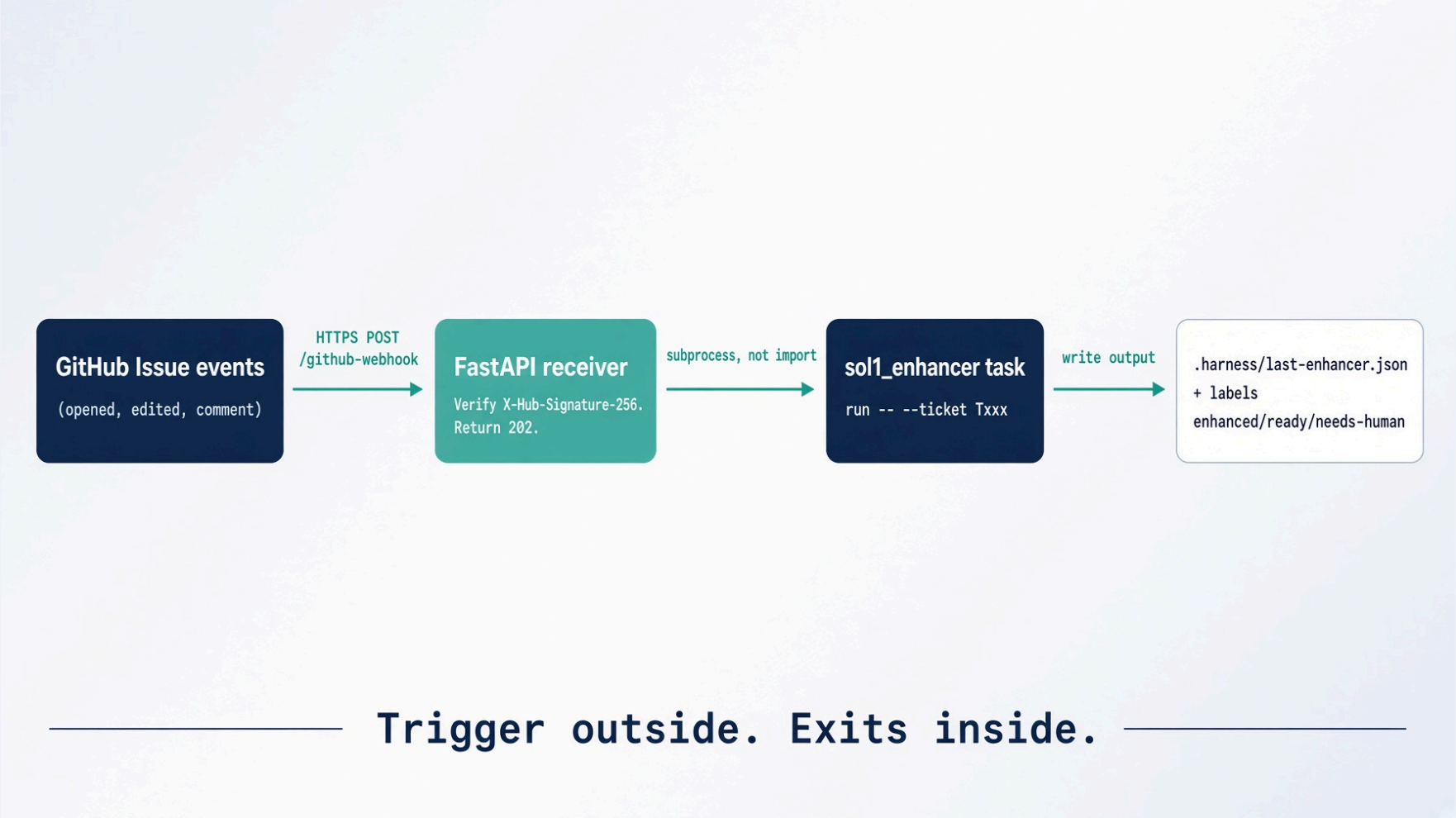
Learning objectives

- Split receive from work
- Verify HMAC on the raw ALB body
- Persist `X-GitHub-Delivery` before enqueue
- Keep the receiver on a private subnet behind ALB
- Inject secrets. Never bake tokens in the image
- Map this pattern back to `s_ext_1_webhook.webhook.py`

Starting architecture



Same rule as extra credit



On Fargate the teal box becomes "verify + SQS". The navy `task run` box becomes the worker service.

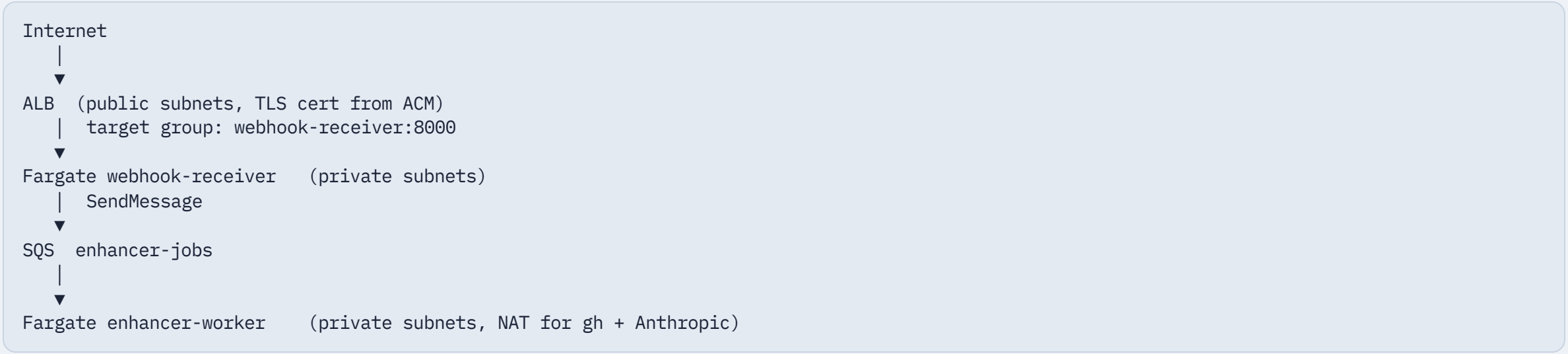
Assumption. Say it out loud.

This deck is a production mapping of extra credit 1. The repo does **not** ship Terraform.

If you implement it, keep:

- `verify_signature` byte-for-byte from `webhook.py`
- `202` before work
- marker skip
- `AGENT_MAX_ATTEMPTS`
- subprocess, not import
- no merge tool

Network. Private tasks. Public ALB.



Assign public IPs only if you skip NAT. Do not. The worker holds a token.

Receiver. Keep extra credit 1's contract.

```
body = await request.body()
verify_signature(body, x_hub_signature_256)   # 401 / 503
if already_seen(x_github_delivery):           # DynamoDB Put with cond
    return JSONResponse({"status": "duplicate"}, 200)
sqs.send_message(QueueUrl=..., MessageBody=body.decode(),
                 MessageAttributes={"event": x_github_event})
return JSONResponse({"status": "accepted"}, 202)
```

Target p95 under 2 seconds. Alert if it creeps toward GitHub's 10-second budget.

Idempotency. Delivery id is the key.

GitHub retries. SQS is at-least-once. Two belts:

1. DynamoDB item `delivery_id` with a condition `attribute_not_exists`
2. Worker lock per ticket (the extra-credit `LOCK_DIR` idea, now a DynamoDB lock or SQS message group id)

FIFO SQS with `MessageGroupId = issue_number` serializes one ticket. Standard SQS plus a lock is the other shape. Pick one.

Worker. Same subprocess as the Droplet.

```
# enhancer-worker: long-running consumer
ticket = ticket_id_from_issue(payload)
run_sol1(ticket)   # call_sol1.py, AGENT_BACKEND, timeout 900
```

Image contains `task`, the lab checkout, and the chosen backend CLI. Task definition `ulimits` and ephemeral storage must fit a git clone of the CRM.

Scale the worker on SQS depth, not on ALB request count.

Secrets. Three, maybe four.

SECRET	WHO READS IT
`GITHUB_WEBHOOK_SECRET`	receiver only
`GITHUB_TOKEN`	worker (labels, comments, issue body)
`ANTHROPIC_API_KEY`	worker, when backend is claude / agent-sdk / deep-agents
`OPENAI_API_KEY` / `XAI_API_KEY`	worker, matching `AGENT_BACKEND`

Task definition: `secrets` from Secrets Manager. Rotate the webhook secret on a 90-day calendar. Accept both during the window.

IAM. Two roles, not one.

Receiver task role: `sqs:SendMessage`, `dynamodb:PutItem`, `secretsmanager:GetSecretValue` for the webhook secret only.

Worker task role: `sqs:ReceiveMessage/DeleteMessage`, `dynamodb:Get/Put` for locks, `secretsmanager` for token and model keys. No `sqs:SendMessage` on this queue.

Execution role: pull from ECR, write CloudWatch. That is not the task role.

Health, timeouts, drain

ALB health check: `GET /health` on the receiver. Same payload as extra credit 1.

Receiver `stopTimeout` short. Worker `stopTimeout` long enough to finish one poll or nack the SQS message.

`proxy_read_timeout` on the Droplet was 120s. Here the ALB idle timeout can stay at 60s because work is not on that socket.

Backend choices on Fargate

BACKEND	ON FARGATE?
`agent-sdk` / `deep-agents`	best fit. Python in the image, key in Secrets Manager
`claude` CLI	works if you install it at build. Headless `-p`
`opencode`	works if the image has it
`codex`	possible. Watch the sandbox flags from `bin/role.sh`
`grok`	poor fit. Trust prompt, local shims. Keep on a Droplet

Observability. The 2 a.m. test.

Ship `last-webhook.json` fields to CloudWatch as structured logs: delivery, issue, ticket, backend, returncode.

Alarm on:

- receiver 4xx rate (bad signatures)
- receiver p95 latency
- SQS age of oldest message
- worker non-zero returncode
- `gave-up` comments

If you cannot read the last score, you cannot debug unattended.

Security extras GitHub already gives you

Allow the ALB security group from GitHub webhook IP ranges, or put a WAF with a GitHub Hookshot user-agent plus HMAC (HMAC is the real gate). IP allowlists drift. HMAC does not.

Reject bodies over 1 MB, same as the nginx `client_max_body_size`.

Never log the raw secret or the `Authorization` header.

Walkthrough. Twelve steps.

1. ECR repo, image from extra credit 1 plus `call_sol1.py`
2. VPC: 2 public, 2 private, NAT
3. ACM cert on the hostname GitHub will call
4. ALB + target group + HTTPS listener
5. SQS queue (FIFO if you want per-issue serialization)
6. DynamoDB table `deliveries` (pk: delivery_id)
7. Secrets Manager entries
8. Receiver service, desired count 2
9. Worker service, desired count 1, scale on queue depth
10. Point the CRM fork webhook at `https://agents.example.com/github-webhook`
11. Open `[T900] ...`. Watch 202, then a worker log, then a marked comment
12. Comment `LGTM` on a green rubric. Next message should set `ready`

Troubleshooting

SYMPTOM	CAUSE	FIX
GitHub delivery timeout	work on the request thread	enqueue, then 202
401 from ALB	body decoded then re-encoded	HMAC the raw bytes ALB gave you
Duplicate polls	no delivery-id table	DynamoDB condition put
Worker cannot `gh`	no NAT, or token missing	private subnet plus NAT. Secret on the worker
Scale to zero, cold 15s	receiver scaled to 0	keep desired count >= 1, or accept the miss

Recap

What changed. The doorbell is an ALB. The waiting room is SQS. The loop is still `sol1_*`.

What did not change. HMAC. 202. Marker. Three exits. No merge.

Takeaways

1. Validate on the edge. Process on a bus.
2. Size the worker for the model, the receiver for HMAC.
3. Two IAM roles.
4. Grok still belongs on a Droplet.
5. If you cannot read the last score, you cannot debug at 2 a.m.

Closing line. Fargate is a bigger doorbell. It is still not the loop.